



Brainbox AI Training Guide & API Reference

How to teach Brainbox your company's knowledge (documents, pasted answers and support tickets), who gets to see what, and the complete REST API that customers can use to train Brainbox and put it on their own websites.

BRAINBOX API

1.0.0 · base path /api

WRITTEN FOR

Owners, staff &
developers

DATE

8 October 2026

Part A explains training in plain language. Part B is the developer reference: you can give it to a customer's developers as is. Every endpoint, field and limit here was checked against the brainBox source code.

Contents

PART A • TRAINING GUIDE for owners, staff and customers' admins

1	How Brainbox learns	From a PDF to an answer, in plain words	3
2	Ways to train	Dashboard, API and answering gaps; file types and limits	5
3	Connecting a support system	Tickets and our responses, fields, pages, re-sync	8
4	Audiences: who sees what	public, customer, vendor, internal, admin	12
5	Knowledge gaps and staff alerts	Unanswered questions, emails, "Answer & train"	13
6	Staff roles	owner, admin, trainer, viewer; invites; first owner	16
7	Keys explained	Publishable, secret, staff login, admin token	18
8	Best-practice checklist	Good training data in one page	20

PART B • API REFERENCE for developers

9	API basics	Base URL, auth headers, tenants, errors, CORS, timeouts	21
10	Endpoint reference	Every endpoint, request, response and errors	24
11	Quick-start recipes	curl, JavaScript and Python, step by step	53
12	Using the SDKs	sdk-web, sdk-react, React Native, Node, Python	59
13	Server admin appendix	Environment variables, CLI, deployment order	62
14	Glossary	The vocabulary in plain words	65

HOW TO READ THIS DOCUMENT

Part A (sections 1 to 8) is for everyone who trains Brainbox: you don't need to be a developer. "For developers" boxes add technical detail you can skip. Part B (sections 9 to 14) is the technical reference for the people who connect Brainbox to a website, an app or a support system.

ABOUT SECRETS IN THIS DOCUMENT

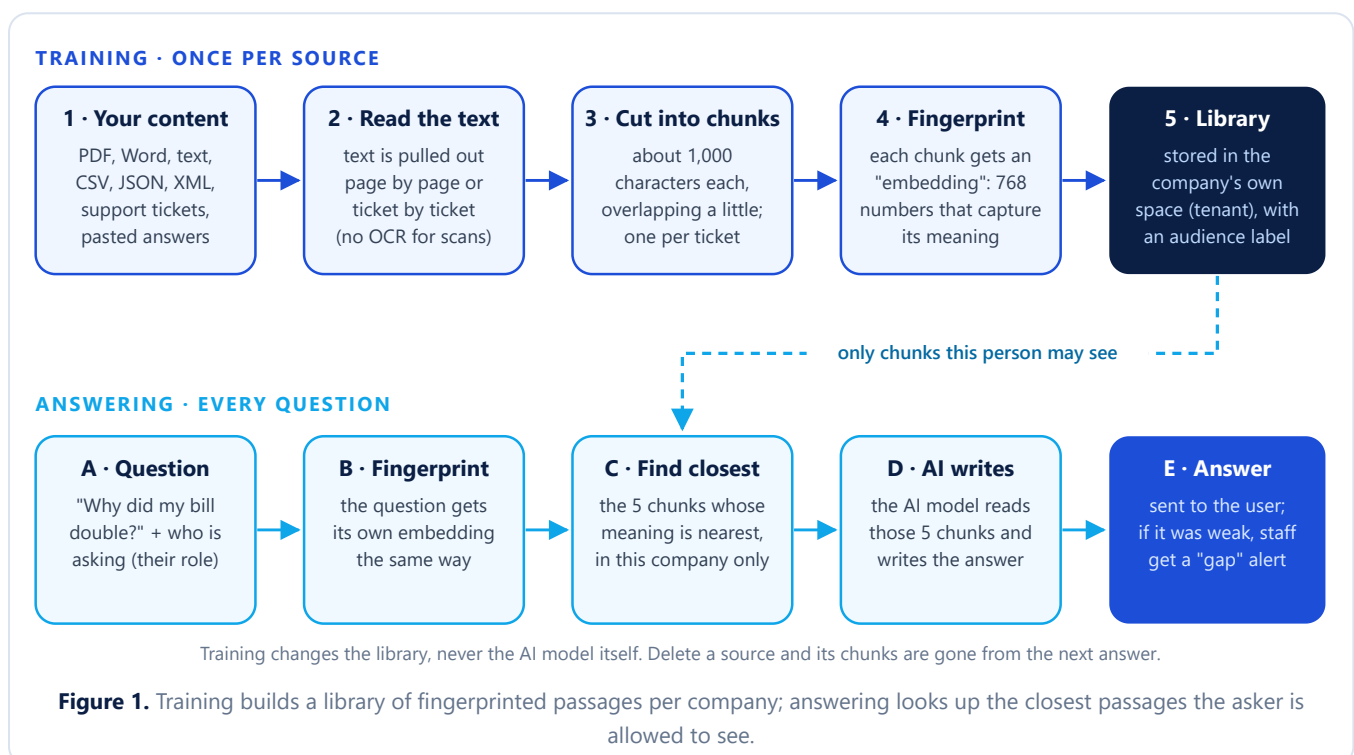
No real key, password or token appears here. Keys are written as placeholders: `pk_live_xxx` (publishable), `sk_live_xxx` (secret), `<staff-jwt>` (a staff login token) and `<admin-token>`. The only real address used is the public brainBox server `https://port.smartpowerbilling.com`.

Training guide

For owners, staff and customers' admins. No programming needed.

1 How Brainbox learns

Brainbox does not "study" your documents the way a person does, and it does not change the AI model. It builds a searchable library of short passages from your content and, for every question, hands the AI the few passages that fit best. The AI then writes the answer from them.



The five training steps in plain words

- 1 Upload.** You give Brainbox content: a file, a piece of text you type, or the address of a system (for example your support desk) that it can read from.

Each upload becomes one **training source** that you can see in the list, rename, relabel, re-sync or delete.

- 2 Read the text.** Brainbox pulls the plain text out of the file. Pictures are ignored, and a scanned PDF (a photo of paper) has no text, so it can't be read.

PDF via pypdf, Word via python-docx, XML via defusedxml; text files are decoded as UTF-8 (with fallbacks).

- 3 Split into chunks.** Long text is cut into passages of about 1,000 characters (roughly half a page). Neighbouring chunks overlap by about 150 characters so a sentence cut in two is never lost. Each support ticket becomes its own chunk.

Each chunk starts with a label such as `[Manual.pdf - Page 4]` so the AI knows where it came from.

- 4 Fingerprint (embedding).** Each chunk is turned into a list of 768 numbers that describe what it *means*. Passages about the same topic get similar numbers even when they use different words ("bill" and "invoice").

Model: BAAI/bge-base-en-v1.5 (setting `EMBEDDING_MODEL`), stored in PostgreSQL with pgvector.

- 5 Store per company.** Chunks are saved in the company's own space (its **tenant**) together with an **audience** label that says who may see them (section 4).

The same text in two companies is stored twice and never mixed. The same text uploaded twice in one source is stored once.

And when someone asks a question

The question gets a fingerprint too. Brainbox looks up the **5 chunks whose fingerprints are closest**, but only inside the asker's company and only among chunks whose audience the asker's role may see. The AI model receives those 5 chunks plus the question and is told to answer only from them. If it can't, it says so ("I don't have that information yet..."), and your staff get an alert so they can teach it (section 5).

IN PLAIN WORDS

Think of a librarian with an index card for every half page of your documents. When a question comes in, she pulls the five cards that match it best (skipping cards stamped "staff only" if the asker is a customer) and hands them to a writer who drafts the reply. Training adds cards; deleting a source pulls its cards out of the drawer.

FOR DEVELOPERS

- Similarity is cosine distance (0 = same meaning, larger = less related). The closest distance is also used to detect weak answers (default threshold 0.55).
- Answers are cached for 1 hour per company + role + question. The cache is cleared whenever a training job finishes and when a source is relabelled or deleted, so new knowledge shows up straight away.
- The AI model is Ollama (`OLLAMA_MODEL`); OpenAI is used as a fallback only when `USE_OPENAI=true` and a key is set.

2 Ways to train

There are three ways to add knowledge. They all end up in the same library and follow the same audience rules.

1

The dashboard (no code)

The **Training** page of the staff dashboard (the `TrainingPanel`) has three tabs: **Upload files**, **Connect API** and **Paste text**, plus a searchable list of every source with its status, audience, chunk and record counts, and Sync / Delete buttons.

2

The API (automation)

A developer can call the same endpoints from a server with a **secret key**: `/api/train/file`, `/api/train/text` and `/api/train/api-source`. Useful to push new manuals or FAQs automatically, or to sync a support desk every night.

3

Answering gaps

When Brainbox couldn't answer a question, it appears in the **Knowledge gaps** inbox. A trainer types the right answer and clicks **Answer & train**: the answer becomes a new training source and the gap is closed.

Using the Training page step by step

1

Sign in to the staff dashboard with your email and password. You need the **trainer** role or higher to add or change knowledge; viewers can only look.

2

Pick a tab. *Upload files*: drag one or more files in (PDF, XML, TXT, MD, CSV, JSON, DOCX, up to 25 MB each). *Paste text*: give it a name and paste the text. *Connect API*: enter the address of your support system (section 3).

3

Choose the audience ("Audience · who can see this"). The default is **Internal staff**. Choose **Public** only for content you would happily publish on your website.

4

Send it. The source appears in the list as `queued`, then `processing`, then `completed` or `failed`. The list refreshes by itself every few seconds while something is processing.

5

Check the result. A completed source shows how many chunks (and, for tickets, records) it produced. A note such as "3 of 40 pages had no extractable text" means part of the file was skipped; "failed" always comes with a reason.

Supported file types

Type	How Brainbox reads it	Watch out for
PDF	Text is read page by page; every chunk is labelled with the file name and page number. Pages without text are skipped and counted in a note.	Scanned PDFs (images of paper) can't be read: there is no OCR. A PDF with no text at all fails with "No extractable text found". Password-protected PDFs fail unless they open with an empty password.
DOCX (Word)	All paragraphs, then all tables (cells joined with).	Headers, footers, comments and images are ignored. Old .doc files are not accepted: save as .docx . Needs python-docx on the server (otherwise .docx is refused).
TXT, MD	Read as plain text (UTF-8; UTF-16 and Windows encodings are detected). Markdown is treated as text.	Keep one topic per heading; headings help the chunks make sense on their own.
CSV	If most rows look like support tickets (an issue + a response, or a conversation), each row becomes a ticket (section 3). Otherwise rows are grouped 20 at a time, with the header line repeated in every chunk.	Use clear column headers (<code>subject</code> , <code>description</code> , <code>our_response</code> ...). Numbers-only spreadsheets make poor answers.
JSON	Brainbox looks for the list of records (for example under <code>data</code> , <code>items</code> , <code>tickets</code>). Ticket-like records become tickets; anything else is split into readable JSON chunks.	Very deep or code-like JSON is hard for the AI to use; prefer records with readable text fields.
XML	Brainbox finds the repeating element (for example every <code><ticket></code> or <code><item></code>). Ticket-like records become tickets; other records become lines like <code>item/title: ...</code> . Content outside the records (document headers) is kept as separate chunks.	XML with no repeating element is read as one long list of "path: value" lines. Entity tricks are blocked (defusedxml).

Limits at a glance

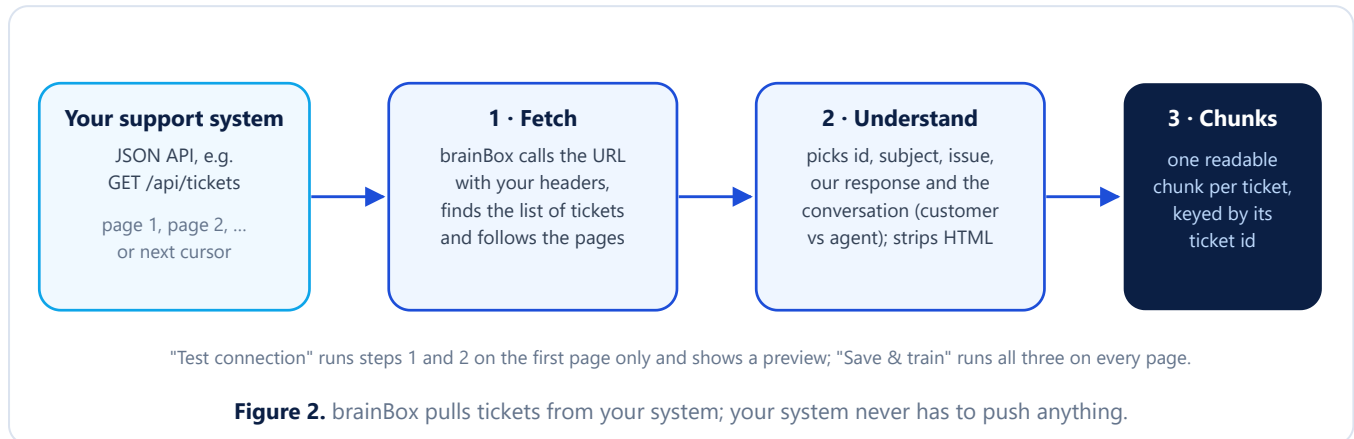
Limit	Value
File size for training	25 MB per file (larger files are refused with "File exceeds the 25 MB limit"). Empty files are refused.
Chunk size	About 1,000 characters with 150 characters overlap (documents and text); up to 2,000 characters per ticket, longer tickets are split into "(part 1/3)"... with the ticket title repeated.
Support-system API	30 seconds per request, 20 MB per response, 10 pages per sync by default (up to 100), JSON responses only.
Pasted text	No size limit in the API (keep it reasonable); the name defaults to the first 60 characters.
Gap answers	Up to 20,000 characters.
Files attached in the chat window	10 MB. These are not training: they are stored with the conversation only and the AI does not read them.

IMPORTANT

- A **server restart** stops jobs that are still running. They are marked failed with "Interrupted by a server restart. Please sync or upload again."
- Uploading a new version of a document does **not** replace the old one: delete the old source first, or both versions will be used.
- Training needs a **secret key** or a staff login. A website's publishable key gets "403 This endpoint requires a secret API key".

3 Connecting a customer-support system

Past support tickets are the most valuable training data: each one is a real customer question together with the answer your team gave. Brainbox can read them straight from your helpdesk's API (Zendesk, Freshdesk, your own system...) and turn every ticket into "customer issue + our response".



What your API should return

Any JSON response that contains a **list of tickets** works. The list can be the whole response, or sit under a key such as `data`, `results`, `items`, `tickets`, `records`, `conversations`, `issues` or `cases` (two levels deep at most, for example `data.tickets`). If it is somewhere else, set **data path** to its location, for example `response.payload.rows`.

Each ticket should contain a customer's issue *and* your answer, either as two fields or as a conversation (a list of messages). Brainbox recognises the usual field names on its own:

Meaning	Field names recognised automatically (first match wins)
Ticket id	<code>ticket_id</code> , <code>ticket_number</code> , <code>ticket_no</code> , <code>case_number</code> , <code>case_id</code> , <code>id</code> , <code>number</code> , <code>key</code> , <code>reference</code> , <code>ref</code> , <code>uuid</code>
Title	<code>subject</code> , <code>title</code> , <code>summary</code> , <code>topic</code> , <code>headline</code> , <code>issue_title</code> , <code>name</code>
Customer issue	<code>customer_issue</code> , <code>issue</code> , <code>question</code> , <code>problem</code> , <code>description</code> , <code>issue_description</code> , <code>customer_message</code> , <code>complaint</code> , <code>query</code> , <code>inquiry</code> , <code>enquiry</code> , <code>request</code> , <code>body</code> , <code>message</code> , <code>content</code> , <code>details</code> , <code>text</code> , <code>comment</code>
Our response	<code>our_response</code> , <code>response</code> , <code>answer</code> , <code>reply</code> , <code>resolution</code> , <code>solution</code> , <code>agent_response</code> , <code>agent_reply</code> , <code>support_response</code> , <code>resolution_notes</code> , <code>resolution_note</code> , <code>response_text</code> , <code>answer_text</code> , <code>reply_text</code> , <code>resolution_text</code> , <code>final_response</code> , <code>workaround</code> , <code>fix</code>
Conversation	<code>messages</code> , <code>comments</code> , <code>replies</code> , <code>conversation</code> , <code>conversations</code> , <code>thread</code> , <code>threads</code> , <code>interactions</code> , <code>conversation_parts</code> , <code>posts</code> , <code>notes</code> (must be a list)
Extra details	<code>status</code> , <code>priority</code> , <code>category</code> , <code>type</code> , <code>tags</code> , <code>product</code> are added at the end of the chunk when present (short values only).

Matching ignores case, spaces, dashes and underscores: `Customer Issue`, `customer_issue` and `customerIssue` are the same. Fields one level down are found too (for example Jira's `fields.summary`). HTML in any field is turned into plain text.

Mapping fields yourself

When the names are unusual, fill in the **mapping**. Every value is a "dot path" inside one ticket: `fields.summary`, `requester.message`, `items.0.body`.

Mapping	Use it for
<code>id_field</code>	The ticket number. Strongly recommended : re-syncs use it to replace a ticket that changed.
<code>title_field</code>	The subject line, shown in the chunk header.
<code>question_field</code>	The customer's issue.
<code>answer_field</code>	Your team's response or resolution.
<code>messages_field</code>	The list of messages in the conversation.
<code>extra_fields</code>	A list of other fields to add (replaces the default status / priority / category / type / tags / product).

Conversation threads

For each message Brainbox finds the text (`body`, `text`, `message`, `content`, `html_body` ...) and decides whether the **customer** or an **agent** wrote it, in this order:

1. yes/no flags such as `is_agent`, `is_staff`, `is_admin`, `is_customer`, `incoming`, `private: true`;
2. a role-like field (`role`, `author_type`, `sender_type`, `from`, `direction`, `source` ...) or an `author` / `user` object whose value contains words like *agent*, *admin*, *staff*, *support*, *team*, *bot*, *outbound* (agent) or *customer*, *user*, *client*, *requester*, *contact*, *inbound* (customer);
3. otherwise: the first message is the customer, later messages by the same author id are the customer and anyone else is an agent; failing that, turns simply alternate.

If the first message only repeats the issue, it is left out so the chunk doesn't say it twice.

Pages: getting more than the first page

Type	How it works	Settings
<code>none</code>	One request; whatever comes back is used. (Default.)	—
<code>page</code>	Calls with <code>?page=1</code> , <code>2</code> , <code>3</code> ... until a page is empty, the same page comes back twice (the API ignores the parameter), or the page limit is reached. Starts from the page number in your query if you set one.	<code>page_param</code> (default <code>page</code>), <code>max_pages</code> (default 10, max 100)
<code>cursor</code>	Reads a "next" token or link from each response and sends it back as a parameter. If the value is a full URL (a "next" link) it is called as is. Stops when there is no new cursor.	<code>cursor_path</code> (auto: <code>next_cursor</code> , <code>meta.next_cursor</code> , <code>pagination.next_cursor</code> , <code>next_page_token</code> , <code>links.next</code> , <code>next_page_url</code> , <code>next</code> , <code>paging.next</code> , <code>cursor</code> ...), <code>cursor_param</code> (default <code>cursor</code>), <code>max_pages</code>

Worked example

Your helpdesk returns this for `GET https://help.example.com/api/tickets` (with your helpdesk's token in a header):

```
{
  "data": {
    "tickets": [
      { "ticket_id": "T-1042", "subject": "Meter shows double usage",
        "description": "My meter reading jumped from 350 kWh to 700 kWh this month but nothing changed at home.",
        "status": "solved", "priority": "high",
        "messages": [
          { "author_type": "customer", "body": "My meter reading jumped from 350 kWh to 700 kWh this month but nothing changed at home." },
          { "author_type": "agent", "body": "<p>Thanks! We checked the meter remotely: the reading was entered twice.</p>" },
          { "author_type": "agent", "body": "We reversed the duplicate reading and issued a corrected bill." } ] },
      { "ticket_id": "T-1043", "subject": "How do I change my billing email?",
        "description": "I want invoices sent to accounts@example.com.",
        "our_response": "Go to Profile > Billing > Invoice email, enter the new address and click Save. Future invoices go to the new address.",
        "status": "solved" }
    ]
  },
  "meta": { "next_cursor": "eyJwYWdlIjoyfQ" }
}
```

With no data path and no mapping, Brainbox finds the list at `data.tickets`, reports the detected fields `ticket_id`, `subject`, `description`, `status`, `priority`, `messages`, and produces these two chunks (output of the real normalizer):

```
Support ticket #T-1042: Meter shows double usage
Customer issue: My meter reading jumped from 350 kWh to 700 kWh this month but nothing changed at home.
Conversation:
Agent: Thanks! We checked the meter remotely: the reading was entered twice.
Agent: We reversed the duplicate reading and issued a corrected bill.
status: solved
priority: high
```

```
Support ticket #T-1043: How do I change my billing email?
Customer issue: I want invoices sent to accounts@example.com.
Our response: Go to Profile > Billing > Invoice email, enter the new address and click Save. Future invoices go to the new address.
status: solved
```

With `pagination: {"type": "cursor"}` Brainbox would then call the URL again with `?cursor=eyJwYWdlIjoyfQ` (found automatically at `meta.next_cursor`).

Test connection first

Test connection fetches the first page only, trains nothing and shows: whether it worked, the HTTP status, how many records it found, where it found them (`data_path`), the field names of the first record, how many records it had to skip, and a preview of the first 5 chunks exactly as they will be stored. Fix the mapping until the preview reads well, then click **Save & train**.

IN PLAIN WORDS

If the preview says "Records were found but none had a recognizable issue/response", Brainbox found your tickets but couldn't tell which field is the question and which is the answer: set **question field** and **answer field**. If it says "No records found", set the **data path** to where the list of tickets is.

Re-sync: keeping it up to date

Click **Sync** on the source (or call `POST /api/train/sources/{id}/sync`) to fetch everything again. Nothing syncs automatically: if you want a nightly sync, have a server call that endpoint on a schedule.

What happened to a ticket	What a sync does
Unchanged	Skipped: same text, nothing is re-processed.
Changed (new reply, edited text)	The new version is stored and the old version of that ticket id is removed. Without an id, the old version stays next to the new one.
New	Added.
No longer returned by the API	Kept. A sync never deletes tickets it didn't see (the API may simply return the latest 10 pages). To start clean, delete the source and add it again.

IMPORTANT

- Only one sync per source at a time: a second sync gets "This source is already syncing" (unless the first has been stuck for more than 30 minutes).
- The address, headers and mapping of a saved source can't be edited (only its name and audience). To change them, delete the source and connect again.
- Headers and query values whose name looks secret (containing auth, token, key, secret, password, cookie, session, signature, credential or bearer) are hidden as `....` when sources are listed, but they are stored on the brainBox server so it can sync. Use a read-only helpdesk token.
- Choose the audience carefully: tickets usually contain customer details. The AI is instructed never to reveal other customers' names or contact details, but **internal** is the safe default.

FOR DEVELOPERS

`source_type: "api_generic"` treats records as generic data instead of tickets: each becomes `Record #id: title` followed by every field as `path: value` lines. POST requests are supported (`method: "POST"` with a `JSON body`). Responses must be JSON; redirects are followed.

4 Audiences: who can see what was trained

Every training source carries one **audience** label. Every question is asked with one **role**. Brainbox only searches the chunks whose audience that role may read, so restricted content never even reaches the AI model.

Audience	Meant for	Role asking	pub	cust	vend	int	adm
public	Anyone, including anonymous website visitors.	admin	✓	✓	✓	✓	✓
customer	Logged-in customers and portal users.	internal	✓	✓	✓	✓	–
vendor	Suppliers and vendors.	customer / portal	✓	✓	–	–	–
internal	Your staff. Default for new sources.	vendor	✓	–	✓	–	–
admin	Administrators only (sensitive material).	public	✓	–	–	–	–

Where the asker's role comes from

How the question arrives	Role used
Website widget with a publishable key	Always public , whatever the page sends. A browser can't be trusted to say who it is.
A server with a secret key (Odoo add-on, your backend)	The <code>user_role</code> the server sends: <code>admin</code> , <code>internal</code> , <code>customer</code> (or <code>portal</code>), <code>vendor</code> or <code>public</code> . A missing or unknown role means public , so a server that wants staff answers must send <code>internal</code> explicitly.
A logged-in staff member (staff token)	Treated like a secret key: the role sent with the question (default public).

Defaults you should know

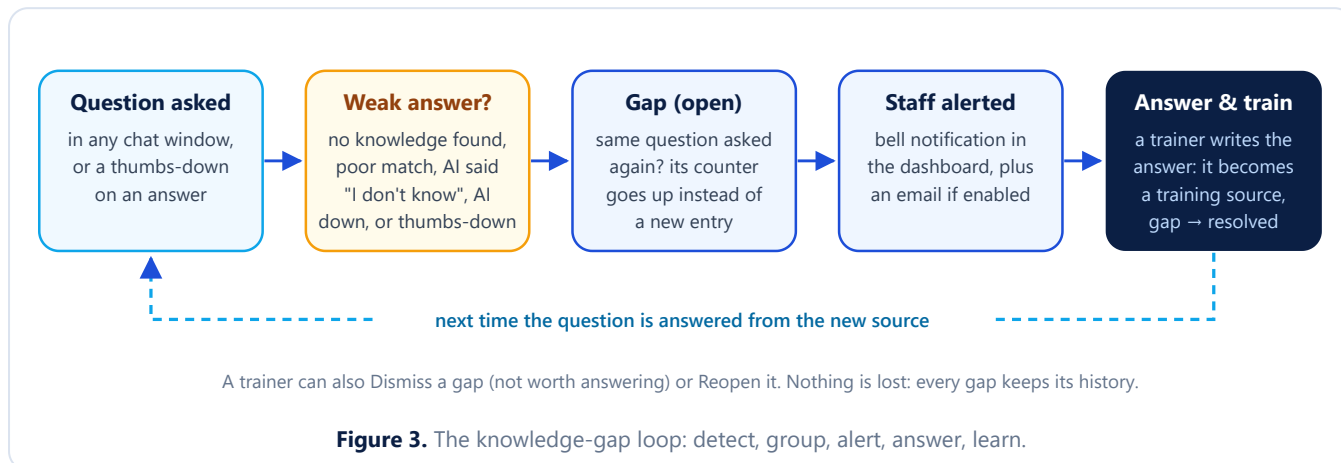
- **Files, pasted text and connected APIs** default to **internal** when no audience is chosen.
- **Gap answers** ("Answer & train") default to **public**, because they usually answer a customer's question. Change the picker before saving if the answer is internal.
- **Old content** trained before audiences existed, and anything sent through the low-level `/api/ingest` endpoint without an audience, has no label. It is treated as the server setting `LEGACY_DOC_AUDIENCE`, which is **public** unless the server administrator changed it. Set it to `internal` to hide all unlabelled content from the public.
- **Audiences protect answers, not the training list:** anyone with a secret key or a staff login can list every source of the company. And they only work if the role is right: a server that sends every user as `admin` shows everything to everyone.

How to relabel a source

In the Training list, click the source's audience pill and pick another audience (trainer role or higher). Through the API: `PATCH /api/train/sources/{id}` with `{"audience": "customer"}`. Every chunk of the source is relabelled at once and cached answers are cleared, so the change applies to the very next question.

5 Knowledge gaps and staff alerts

Brainbox notices when it could not answer properly, writes the question down in a **Knowledge gaps** inbox and tells your staff. Answering a gap teaches Brainbox, so the same question is answered next time. This is the fastest way to improve it.



When a question is recorded as unanswered

After every answer Brainbox checks, in this order, whether it was weak. The first matching reason is recorded:

Reason	Shown as	What it means
<code>llm_unavailable</code>	the AI model was unavailable	The AI model didn't respond, so a fallback message was sent. Usually a server problem, not missing knowledge.
<code>no_context</code>	no matching knowledge	Nothing in the library that this person may see matched at all.
<code>llm_unknown</code>	the assistant didn't know	The answer says it doesn't know. The AI is told to start such answers with "I don't have that information yet"; similar phrases ("I couldn't find...", "not in the knowledge base") are detected too.
<code>low_confidence</code>	low-confidence match	Something was found, but even the closest chunk was far from the question: its distance was above the company's gap threshold (default 0.55 ; 0 = identical meaning, 2 = opposite).
<code>negative_feedback</code>	negative feedback	A user gave the answer a thumbs-down (optionally with a comment).

Admins can change the gap threshold per company in Settings (any number from 0 to 2). Lower it to flag more answers as weak, raise it to flag fewer. A cached answer that was weak the first time still counts as unanswered when it is served again.

Grouping

Questions are grouped per company by their normalized text: lower-case, punctuation removed, spaces collapsed. "How do I reset my PIN?" and "how do i reset my pin" are one gap. A repeat increases its **occurrences** counter and updates "last seen", the asker and the answer given; the status stays as it was, except that a **thumbs-down re-opens a resolved gap**. Gaps are sorted by when they were last asked, so the most current problems come first.

Who is alerted, and how

Event	Who	How
A new gap (first time a question goes unanswered)	Every active staff member (all roles)	In-app notification "New unanswered question", plus an email if the company's Notify on gap setting is on.
Every new thumbs-down	Every active staff member	"Negative feedback on an answer", with the comment; email if Notify on feedback is on.
A training source failed	Admins and owners	"Training failed: <name>" with the reason; email.
An invited colleague joined	Admins and owners	In-app only.

- Each staff member chooses for themselves in **Account**: in-app notifications on/off, emails on/off. People who haven't accepted their invitation yet get nothing.
- Emails need the server's email (SMTP) to be configured. Without it, only in-app notifications are created.
- To avoid floods, each company sends at most **20 alert emails per hour** by default (Settings → Gaps & alerts, 0 to 1000; 0 turns alert emails off). Beyond that, emails are skipped but in-app notifications are still created. Invitations and password resets are never limited.
- Repeats of the same gap don't alert again: only its counter goes up.

Answering a gap ("Answer & train")

- 1 **Open the gap** from the bell or the Gaps inbox. You see the question, the reason, how often it was asked, by whom (name and role, when the website or Odoo sent them), the answer Brainbox gave and any feedback comment.
- 2 **Write the correct answer** in your own words, as you would to a customer.
- 3 **Choose the audience** ("Who can see this answer", default **public**) and decide on **Also resolve similar open questions** (ticked by default in the dashboard; off by default in the API).
- 4 **Click Answer & train**. Brainbox creates a text training source named `Answer: <first 60 characters of the question>` containing `Question: ...` and `Answer: ...`, trains it, and marks the gap **resolved** with your name.

"Also resolve similar open questions" closes the other **open** gaps whose questions share at least 60% of their meaningful words with this one (common words like "how", "the", "my" are ignored) and links them to the same answer.

A trainer can also **Dismiss** a gap (spam, out of scope), **Reopen** it, or mark it resolved with a note (for example after uploading a better manual).

Thumbs-down feedback

Any chat window can send a rating for an answer with `POST /api/chat/feedback` (a publishable key is enough): `up` or `down`, with an optional comment of up to 2,000 characters. A thumbs-down creates or bumps a

`negative_feedback` gap for the question and alerts staff. Changing your mind and rating the same answer again updates the rating; a second "down" on the same answer does not alert again.

IMPORTANT

The ready-made chat widgets (sdk-web and the React `ChatWidget` / `ChatPanel`) don't show thumbs buttons yet. Feedback works today through the API (see the recipe in section 11), and staff can read the ratings in the Conversations view.

6 Staff roles

The staff dashboard has its own accounts: each person signs in with their email and a password, belongs to one company, and has one of four roles. No API key is needed to use the dashboard.

What they can do	viewer	trainer	admin	owner
See the overview, gaps, conversations, training sources, staff list and settings	✓	✓	✓	✓
Their own profile, password and notification choices	✓	✓	✓	✓
Train: upload files, paste text, connect APIs, sync, relabel, rename, delete sources	–	✓	✓	✓
Answer, dismiss, reopen and annotate knowledge gaps	–	✓	✓	✓
Invite, change, deactivate and remove trainers and viewers	–	–	✓	✓
Change company settings (name, alerts, gap threshold, chat-widget look)	–	–	✓	✓
Create and revoke API keys	–	–	✓	✓
Receive "training failed" and "colleague joined" alerts	–	–	✓	✓
Invite, change and remove admins and owners	–	–	–	✓

Admins can manage only roles below their own, so an admin can't promote anyone to admin. A company must always keep at least one active owner: demoting, deactivating or removing the last one is refused ("A tenant must keep at least one active owner").

Inviting staff

- 1 Staff → Invite.** An admin or owner enters the email, an optional name and the role.
- 2 The invitation** is emailed with a link valid for **7 days**. If the server can't send email, the dashboard shows the link to copy and send yourself. "Resend invite" makes a new link.
- 3 The colleague** opens the link, chooses a password (at least 8 characters) and is signed in. Admins get a "joined" notification.

- One email address is one account, in one company. Inviting an address already used anywhere is refused.
- Forgotten password: "Forgot password" emails a reset link valid for **1 hour** (at most one email every 2 minutes). The answer is always "ok", so nobody can test which emails exist.
- A sign-in lasts **12 hours**. After 10 wrong passwords in 15 minutes for the same email and computer, sign-in is blocked for a while ("Too many failed attempts").
- Deactivating or removing someone, or changing their role, takes effect on their very next click.

The very first owner

A new company has nobody to send the first invitation, so the server administrator creates the first owner, either on the server:

```
python -m app.cli staff create --tenant acme --email owner@acme.com --role owner --name "Ada Lovelace"  
# prints an invite link (valid 7 days). Add --password to type a password instead.
```

or over HTTP with the admin token (POST `/api/admin/staff`, section 10). From then on, the owner invites everybody else from the dashboard.

FOR DEVELOPERS

Invite and reset links point to `DASHBOARD_URL/#/accept-invite?token=...` and `.../#/reset-password?token=...`. When `DASHBOARD_URL` isn't set, the browser's Origin header is used; emails are only sent when an absolute link can be built.

7 Keys explained

Four kinds of credentials open brainBox. Each one says **who** is calling and **which company** they act for. Choosing the right one is the most important security decision when you connect Brainbox to something.

PUBLISHABLE KEY · PK_LIVE_...

For websites and apps

Safe to put in a web page or mobile app. It can only chat: ask questions, keep conversations, attach files to a chat, send feedback and read the widget's look. Every user is treated as **public**, so only public knowledge is used.

SECRET KEY · SK_LIVE_...

For servers only

Can do everything for its company: chat with a trusted role, train, list and delete sources, read task status. **Never** put it in a browser, a mobile app, a public repository or an email.

STAFF LOGIN · <STAFF-JWT>

For people in the dashboard

What you get when you sign in with email and password. Works like a secret key for training (trainer role or higher to change things) and also opens the staff-only features: gaps, reports, staff, settings, keys. Expires after 12 hours.

ADMIN TOKEN · <ADMIN-TOKEN>

For the server administrator

One long password set on the brainBox server (`BRAINBOX_ADMIN_TOKEN`). It manages keys for **every** company and can create a company's first owner. It can't chat or train.

Can it...	Publishable	Secret	Staff login	Admin token
Chat, list and read conversations, upload into a chat	yes (as public)	yes (trusted role)	yes	no
Send feedback, read widget appearance	yes	yes	yes	no
Train, list / relabel / delete sources, ingest	no (403)	yes	read: viewer · change: trainer+	no
Gaps, reports, notifications, settings, staff, keys	no (403)	no (403)	yes by role	no
Create / revoke keys for any company, first owner	no	no	own company only (admin+)	yes

Getting and managing keys

- **Dashboard:** Settings → API keys (admin or owner). Give the key a name ("Website widget", "Odoo server"), pick publishable or secret, and optionally an expiry date. **The key is shown once:** copy it immediately. Revoke it there when it is no longer needed or may have leaked.
- **Server:** `python -m app.cli keys create --tenant acme --type secret --name "Odoo server"`, `keys list`, `keys revoke 42`.
- brainBox stores only a fingerprint (hash) of each key, so a lost key can't be recovered: create a new one and revoke the old.

- Expired and revoked keys get `401 Invalid or missing API key` immediately.

NEVER PUT A SECRET KEY IN A BROWSER

Anything in a web page can be read by every visitor (View Source takes two seconds). A leaked secret key lets a stranger read your whole training list, delete it, or train Brainbox with false answers. If a website needs more than public answers, its **own server** must call brainBox with the secret key (a "proxy"), after checking who the user is. This is exactly what the Odoo add-on does.

IN PLAIN WORDS

The publishable key is the shop's front-door bell: anyone can ring it and talk to reception. The secret key is the master key to the back office. The staff login is a personal badge with the right floors on it. The admin token is the building manager's key ring.

8 Best-practice checklist for good training data

Brainbox can only be as good as what it reads. These habits make the biggest difference.

Content

- ☐ Start with what customers ask most: FAQs, prices and plans, billing, how-to guides, policies (refunds, cancellations, opening hours).
- ☐ One topic per document or per section, with clear headings. A chunk is about half a page: it should make sense on its own.
- ☐ Write answers as full sentences. "Refunds: 5d" is hard to use; "Refunds are paid back within 5 working days" is easy.
- ☐ Use the words your customers use ("bill" and "invoice", "meter" and "reading").
- ☐ Prefer text PDFs and Word files. If you can't select the text in a PDF, it's a scan: Brainbox can't read it.
- ☐ Remove outdated versions: delete the old source before uploading the new one.
- ☐ Avoid huge raw exports (logs, number tables): they crowd out useful passages.

Audience and safety

- ☐ Choose the audience every time. Leave internal procedures, prices you don't publish and anything personal as **internal** or **admin**.
- ☐ Use **public** only for what you would publish on your website.
- ☐ Strip passwords, bank details and personal data before uploading.
- ☐ Support tickets: connect with a **read-only** helpdesk token, keep them **internal** unless you are sure, and map the ticket id so re-syncs stay clean.

Keep improving

- ☐ Check the Knowledge gaps inbox weekly and answer the most-asked questions first.
- ☐ Use "also resolve similar" to close duplicates in one go.
- ☐ Look at the overview's answer rate and top questions every month.
- ☐ After a big upload, ask Brainbox a few real questions as a customer would.
- ☐ Re-sync support sources regularly (or automate the sync call).
- ☐ Check failed sources: the reason is always shown.

THE ONE-SENTENCE RULE

Give Brainbox the answers your best support agent would give, written down clearly, labelled with who may hear them, and keep the gap inbox empty.

API reference

For developers. Hand this part to a customer's developers to train Brainbox and use it from their own websites and servers.

9 API basics

brainBox is a JSON-over-HTTPS REST API (FastAPI). All endpoints live under `/api`. There is no SDK requirement: anything that can send HTTP requests can use it.

Base URL

```
https://port.smartpowerbilling.com           # the hosted brainBox server
https://port.smartpowerbilling.com/api/ ...   # every endpoint below
```

Self-hosted servers use their own address (locally `http://localhost:8000`). FastAPI's generated pages `/docs` and `/openapi.json` are also served; this document is the authoritative description.

Authentication

Credential	Send it as	Notes
API key	Authorization: Bearer pk_live_XXX or X-API-Key: pk_live_XXX	Publishable (pk_) or secret (sk_). X-API-Key wins if both headers are sent. Keys never go in the URL.
Staff login	Authorization: Bearer <staff-jwt>	The access_token from POST <code>/api/staff/login</code> . Valid 12 h (expires_in seconds). Recognised automatically (a JWT starts with eyJ).
Admin token	X-Admin-Token: <admin-token>	Only for <code>/api/admin/*</code> and the legacy <code>/api/auth/api-key</code> .

Each endpoint in section 10 shows one of these access badges:

Badge	Who may call
PUBLIC	No credentials.
ANY KEY	Publishable key, secret key or staff login. With a publishable key the role is forced to public .
SECRET KEY / STAFF	Secret key, or a staff login. Staff: viewer may call GET, everything else needs trainer or higher. Publishable keys get 403.
STAFF · ROLE+	Staff login with at least that role. API keys (even secret) get 403 Staff login required.
ADMIN TOKEN	X-Admin-Token header. Independent of API keys.

Tenant resolution

- The **tenant** (company) always comes from the credential: the key's tenant or the staff user's tenant.
- **tenant_id** is optional in every body, query string and form. If you send it and it differs from the credential's tenant: `403 tenant_id does not match API key`.
- Staff-only endpoints take no **tenant_id** at all.
- Legacy mode (`REQUIRE_API_KEY=false` on the server, only during migrations): keys aren't checked, **tenant_id** is required (400 otherwise) and **user_role** is taken from the request as is.

End users: **user_id**, **user_name**, **user_role**

Chat endpoints accept an optional identity for the person chatting. **user_id** (for example `"shop:42"`) stamps new sessions: a stamped session can then only be listed, read or continued with the same **user_id** (otherwise 404, so its existence isn't revealed). **user_name** is shown to staff. **user_role** selects the audiences (section 4) and is only honoured with a secret key or staff login.

IMPORTANT

user_id is a label, not a password: brainBox can't verify it. With a publishable key a visitor can send any **user_id**, and listing sessions **without** a **user_id** returns every session of the tenant. Where conversation privacy matters, let your own server set **user_id** from its login session (proxy mode, section 12) instead of the browser.

Errors

Errors are JSON with a **detail** field. It is a string for errors raised by brainBox, and a list of problems for request-validation errors (422):

```
{ "detail": "This endpoint requires a secret API key (sk_...). Publishable keys (pk_...) can only use chat endpoints." }

{ "detail": [ { "type": "missing", "loc": ["body", "question"], "msg": "Field required", "input": {} } ] }
```

Status	Meaning	Typical detail
200 / 201 / 202	OK / created / accepted (training queued)	–
400	Bad request	"The uploaded file is empty", "Only API sources can be synced"
401	Missing or bad credentials	"Invalid or missing API key", "Invalid or expired session. Please log in again."
403	Credential valid but not allowed	"This endpoint requires a secret API key...", "tenant_id does not match API key", "Staff login required", "Your role (viewer) can't do this"
404	Not found (or belongs to another tenant/user)	"Session not found", "Training source not found", "Knowledge gap not found", "Task not found"
409	Conflict	"This source is already syncing", "A tenant must keep at least one active owner"
413	File too large	"File exceeds the 25 MB limit" (training), "File size exceeds 10.0MB limit" (chat uploads)
415	Unsupported file type	"Unsupported file type '.doc'. Allowed: .csv, .docx, ..."
422	Invalid field values	Validation list, or a message such as "audience must be one of: ..."
429	Too many failed logins	"Too many failed attempts. Try again in a few minutes."
500	Server error	"Database error", or the internal error text on chat endpoints
503	Admin API disabled	"Admin API disabled: set BRAINBOX_ADMIN_TOKEN on the server"

CORS

Every origin, method and header is allowed, with `allow_credentials=false`: a page on any domain can call the API with its key in a header (keys are never cookies).

Timeouts and performance

- **Chat can take up to about 180 seconds** when the AI model is busy (the server waits up to 180 s for the model). Set client timeouts to at least **200 s** (sdk-web uses 200,000 ms), and make sure any reverse proxy in front of your own servers allows it too. Repeated questions are answered from the cache in milliseconds.
- Training endpoints return **202** immediately; the work happens in the background. Poll `GET /api/train/sources/{id}` (or `/api/ingest/status/{task_id}`) every few seconds until `status` is `completed` or `failed`.
- Support-system fetches time out after 30 s per request.

Versioning

The API is version **1.0.0** (reported by `GET /` and `GET /api/health`) and has no version prefix in the path. Changes are made additively: new optional request fields and new response fields can appear at any time, so clients must ignore fields they don't know. Fields marked "additive" below were added after the first release of an endpoint. Times are ISO-8601 strings in UTC (chat-session times may come without an offset: treat them as UTC). Ids of sources, sessions and tasks are UUID strings; ids of gaps, staff, notifications, keys and messages are integers.

10 Endpoint reference

Every endpoint of brainBox, grouped by purpose. Paths are relative to the base URL. Field tables list every accepted field (checked against the server's request models); "Req." = required. Responses show realistic example values.

Index

Method	Path	Access	Purpose
HEALTH			
GET	/api/health	PUBLIC	Liveness check
GET	/api/health/db	SECRET KEY / STAFF	Database connectivity
GET	/api/health/cache	SECRET KEY / STAFF	Redis cache connectivity
GET	/api/health/vector	SECRET KEY / STAFF	pgvector extension, document count (all tenants) and a test embedding
CHAT AND SESSIONS			
POST	/api/chat	ANY KEY	Ask a question and get the answer
POST	/api/chat/session	ANY KEY	Create an empty session (for example to show it in a list before the first question)
POST	/api/chat/sessions	ANY KEY	List sessions grouped by date
GET	/api/chat/session/{session_id}/messages	ANY KEY	All messages of one session, oldest first
FEEDBACK			
POST	/api/chat/feedback	ANY KEY	Thumbs up / down on an assistant answer
CHAT UPLOADS			
POST	/api/chat/upload/file	ANY KEY	Upload any file (multipart/form-data)
POST	/api/chat/upload/image	ANY KEY	Upload an image (multipart/form-data, field image)
TRAINING			
POST	/api/train/file	SECRET KEY / STAFF	Upload a document to train on (multipart/form-data)
POST	/api/train/text	SECRET KEY / STAFF	Train on pasted text (FAQ answers, policies, notes)
POST	/api/train/api-source/test	SECRET KEY / STAFF	Dry run: fetch the first page, find and normalize the records, return a preview
POST	/api/train/api-source	SECRET KEY / STAFF	Save an API source and train on all its pages (the first sync starts immediately)

Method	Path	Access	Purpose
POST	/api/train/sources/{source_id}/sync	SECRET KEY / STAFF	Fetch an API source again and update its knowledge (section 3, Re-sync)
GET	/api/train/sources	SECRET KEY / STAFF	Every training source of the tenant, newest first, with totals
GET	/api/train/sources/{source_id}	SECRET KEY / STAFF	One source
PATCH	/api/train/sources/{source_id}	SECRET KEY / STAFF	Rename and/or relabel a source
DELETE	/api/train/sources/{source_id}	SECRET KEY / STAFF	Delete a source and all its chunks
INGEST (LOW LEVEL)			
POST	/api/ingest	SECRET KEY / STAFF	Queue raw content for chunking and embedding
GET	/api/ingest/status/{task_id}	SECRET KEY / STAFF	Status of any background task: ingest jobs and training jobs (task_id from a train call)
STAFF AUTHENTICATION			
POST	/api/staff/login	PUBLIC	Sign in with email and password
POST	/api/staff/accept-invite	PUBLIC	Finish an invitation: set the password and sign in
POST	/api/staff/forgot-password	PUBLIC	Email a password-reset link (valid 1 hour) if the account exists
POST	/api/staff/reset-password	PUBLIC	Set a new password with the reset token and sign in
STAFF: OWN ACCOUNT			
GET	/api/staff/me	STAFF · VIEWER+	The signed-in staff user
PATCH	/api/staff/me	STAFF · VIEWER+	Update your name and notification choices
POST	/api/staff/me/password	STAFF · VIEWER+	Change your password
STAFF MANAGEMENT			
GET	/api/staff	STAFF · VIEWER+	All staff of the tenant (including pending invitations), oldest first
POST	/api/staff/invite	STAFF · ADMIN+	Invite a colleague
PATCH	/api/staff/{user_id}	STAFF · ADMIN+	Change someone's role, active flag, name or email alerts
DELETE	/api/staff/{user_id}	STAFF · ADMIN+	Remove a staff member (and their notifications)
POST	/api/staff/{user_id}/resend-invite	STAFF · ADMIN+	New invite link (the old one stops working) and email
KNOWLEDGE GAPS			
GET	/api/reports/gaps	STAFF · VIEWER+	The gap inbox, most recently asked first

Method	Path	Access	Purpose
GET	/api/reports/gaps/{gap_id}	STAFF · VIEWER+	One gap
PATCH	/api/reports/gaps/{gap_id}	STAFF · TRAINER+	Resolve, dismiss or reopen a gap, or set its note
POST	/api/reports/gaps/{gap_id}/answer	STAFF · TRAINER+	Answer & train: creates a text training source from the answer and resolves the gap
NOTIFICATIONS			
GET	/api/notifications	STAFF · VIEWER+	Newest first
POST	/api/notifications/{notification_id}/read	STAFF · VIEWER+	Mark one as read
POST	/api/notifications/read-all	STAFF · VIEWER+	Mark all as read
REPORTS			
GET	/api/reports/overview	STAFF · VIEWER+	Dashboard analytics for the last days days (including today, UTC)
GET	/api/reports/conversations	STAFF · VIEWER+	All conversations of the tenant, most recently active first
GET	/api/reports/conversations/{session_id}	STAFF · VIEWER+	One conversation with gap reasons and user ratings per answer
SETTINGS AND WIDGET CONFIGURATION			
GET	/api/settings	STAFF · VIEWER+	The tenant's settings, with defaults filled in
PUT	/api/settings	STAFF · ADMIN+	Partial update: send only the keys to change; widget is merged key by key
GET	/api/widget-config	ANY KEY	The public chat-widget appearance, so a website can style its widget from the dashboard sett...
API KEYS (DASHBOARD)			
GET	/api/keys	STAFF · ADMIN+	All keys of the tenant (never the raw keys), newest first
POST	/api/keys	STAFF · ADMIN+	Create a key
DELETE	/api/keys/{key_id}	STAFF · ADMIN+	Revoke a key (immediately stops working)
ADMIN (SERVER OPERATOR)			
POST	/api/admin/keys	ADMIN TOKEN	Create a key for any tenant, or import an existing raw key so already-deployed sites keep wo...
GET	/api/admin/keys	ADMIN TOKEN	List keys, optionally for one tenant
DELETE	/api/admin/keys/{key_id}	ADMIN TOKEN	Revoke a key
POST	/api/admin/staff	ADMIN TOKEN	Create a staff user in any tenant, typically a new company's first owner
LEGACY: /API/AUTH			

Method	Path	Access	Purpose
POST	/api/auth/signup	PUBLIC	Creates a legacy user account (not a staff account)
POST	/api/auth/login	PUBLIC	Legacy login
POST	/api/auth/api-key	ADMIN TOKEN	Deprecated key creation; use POST /api/admin/keys

Health

Liveness and dependency checks. GET / returns {"service": "Brainbox AI Backend", "status": "running", "version": "1.0.0"} without authentication.

GET /api/health

PUBLIC

Liveness check. No credentials needed. If you *do* send a key or staff token it is validated, which makes this the ideal **Test connection** call for integrations: an invalid key returns 401 here even though the endpoint is public.

EXAMPLE REQUEST

```
curl https://port.smartpowerbilling.com/api/health -H "Authorization: Bearer pk_live_xxx"
```

RESPONSE 200

```
{ "status": "healthy", "service": "brainbox", "version": "1.0.0",
  "key": { "valid": true, "key_type": "publishable", "tenant_id": "acme" } }
```

Errors: 401 Invalid or missing API key (only when a credential was sent)

`key` is present only when a credential was sent; `key_type` is `publishable`, `secret` or `staff`.

GET /api/health/db

SECRET KEY / STAFF

Database connectivity.

RESPONSE 200

```
{ "status": "healthy", "database": "connected" }
```

On failure still returns 200 with `"status": "unhealthy"` and an `error` text. Same pattern for the two below.

GET /api/health/cache

SECRET KEY / STAFF

Redis cache connectivity.

RESPONSE 200

```
{ "status": "healthy", "cache": "connected" }
```

GET

/api/health/vector

SECRET KEY / STAFF

pgvector extension, document count (all tenants) and a test embedding.

RESPONSE 200

```
{ "status": "healthy", "database": "connected", "pgvector": "installed", "pgvector_version":
"0.7.0",
  "documents": 18234, "embedding": "working", "embedding_dimension": 768 }
```

Chat and sessions

The endpoints a website or app uses. All accept a publishable key. A **session** is one conversation; brainBox stores every question and answer in it.

POST

/api/chat

ANY KEY

Ask a question and get the answer. Creates the session when `session_id` is omitted or unknown.

BODY (JSON)

Field	Type	Req.	Description
question	string	yes	The user's question. Integrations may prepend context ("Context: ... Question: ...").
session_id	string	no	Continue this conversation. Unknown id: a session with that id is created. Omitted: a new UUID is used.
user_id	string	no	External end-user id, e.g. <code>"shop:42"</code> . Stamps new sessions (see section 9).
user_name	string	no	Display name for staff reports.
user_role	string	no	<code>admin internal customer portal vendor public</code> . Trusted only with a secret key or staff login; publishable keys are always <code>public</code> . Missing/unknown = <code>public</code> .
tenant_id	string	no	Defaults to the credential's tenant; 403 if it differs.

EXAMPLE REQUEST

```
{ "question": "Why is my bill twice as high this month?",
  "session_id": "6f1c2a9e-1b7d-4a53-9d0e-2b8f7c41e0aa", "user_id": "shop:42", "user_name": "Ada" }
```

RESPONSE 200

```
{
  "response": "A doubled bill is usually caused by a meter reading entered twice. ... ",
  "reasoning": "Used semantic search and Ollama LLM for response",
  "search_results": [
    { "id": 9311, "content": "Support ticket #T-1042: Meter shows double usage ... ",
      "source": "support_tickets", "file_path": "https://help.example.com/api/tickets",
      "distance": 0.2213 }
  ],
  "session_id": "6f1c2a9e-1b7d-4a53-9d0e-2b8f7c41e0aa",
  "message_id": 5121,
  "user_message_id": 5120
}
```

Errors: **401** bad key · **403** tenant mismatch · **404** Session not found (another tenant's, or stamped with another user_id) · **422** missing question · **500** internal error text

search_results lists up to 5 retrieved chunks (first 500 characters each) that the asker was allowed to see; **distance**: lower is closer. **reasoning** is "Retrieved from cache" for cached answers (1 h per tenant + role + question). Keep **message_id** to send feedback. Can take up to ~180 s: use a 200 s timeout.

POST /api/chat/session

ANY KEY

Create an empty session (for example to show it in a list before the first question).

BODY (JSON)

Field	Type	Req.	Description
title	string	no	Defaults to "Chat <first 8 chars of id>".
user_id / user_name / user_role	string	no	As for /api/chat.
tenant_id	string	no	Defaults to the credential's tenant; 403 if it differs.

EXAMPLE REQUEST

```
{ "title": "Billing question", "user_id": "shop:42", "user_name": "Ada" }
```

RESPONSE 200

```
{ "session_id": "0b3f8a51-2c4e-4f0b-a1f6-7d2e9c5b8a10", "title": "Billing question",  
  "created_at": "2026-10-08T09:12:44.512301", "user_id": "shop:42" }
```

POST /api/chat/sessions

ANY KEY

List sessions grouped by date. With **user_id**: only that user's sessions. Without: **all** sessions of the tenant.

BODY (JSON)

Field	Type	Req.	Description
user_id	string	no	Limit to this end user (recommended).
tenant_id	string	no	Defaults to the credential's tenant; 403 if it differs.

EXAMPLE REQUEST

```
{ "user_id": "shop:42" }
```

RESPONSE 200

```
{ "today": [ { "session_id": "6f1c2a9e- ...", "title": "Why is my bill twice as high this month?",  
  "created_at": "2026-10-08T09:12:44.512301", "user_id": "shop:42" } ],  
  "yesterday": [], "this_week": [], "older": [] }
```

Aliases: **POST** /api/sessions, and **GET** /api/chat/sessions?user_id=&tenant_id= / **GET** /api/sessions (same response). Newest first in each group; a session's title is the first 60 characters of its first question.

GET

/api/chat/session/{session_id}/messages

ANY KEY

All messages of one session, oldest first.

QUERY PARAMETERS

Field	Type	Req.	Description
user_id	query	no	Required to read a session stamped with a user_id.
tenant_id	query	no	As above.

RESPONSE 200

```
{ "session_id": "6f1c2a9e- ...", "title": "Why is my bill twice as high this month?",
  "created_at": "2026-10-08T09:12:44.512301", "user_id": "shop:42",
  "messages": [
    { "id": 5120, "role": "user", "content": "Why is my bill twice as high this month?",
      "created_at": "2026-10-08T09:12:44.6", "user_initials": "U", "metadata": { "context_used":
        false } },
    { "id": 5121, "role": "assistant", "content": "A doubled bill is usually ... ",
      "created_at": "2026-10-08T09:12:51.2", "user_initials": "A", "metadata": { "context_used":
        true } } ] }
```

Errors: **404** Session not found

Alias: GET /api/session/{session_id}/messages .

Feedback

POST

/api/chat/feedback

ANY KEY

Thumbs up / down on an assistant answer. **down** creates or bumps a **negative_feedback** knowledge gap and alerts staff. Rating the same answer again updates it.

BODY (JSON)

Field	Type	Req.	Description
session_id	string	yes	The session of the answer.
message_id	integer	no	The assistant message id (message_id from /api/chat). If you pass the user's question id instead, the answer that followed it is rated. Omitted: the session's last answer.
rating	string	yes	up or down .
comment	string	no	Up to 2,000 characters.
user_id	string	no	Owner of the session, if stamped.
tenant_id	string	no	Defaults to the credential's tenant; 403 if it differs.

EXAMPLE REQUEST

```
{ "session_id": "6f1c2a9e- ...", "message_id": 5121, "rating": "down",
  "comment": "This did not explain the estimated reading." }
```

RESPONSE 200

```
{ "ok": true }
```

Errors: **404** Session not found / Message not found · **422** rating not up/down, comment too long · **500** Could not save feedback

Chat uploads

Attach a file or image to a conversation. **Uploads are not training:** the file is stored on the server, a message "Uploaded file: name" is added to the session, and the AI does not read it. To teach Brainbox a document use `/api/train/file`.

POST `/api/chat/upload/file`

ANY KEY

Upload any file (multipart/form-data).

FORM FIELDS (MULTIPART)

Field	Type	Req.	Description
<code>file</code>	file	yes	Max 10 MB.
<code>session_id</code>	string	no	Attach to this session (created if unknown).
<code>user_id</code> / <code>user_name</code> / <code>user_role</code>	string	no	As for chat.
<code>tenant_id</code>	string	no	As above.

EXAMPLE REQUEST

```
curl -X POST https://port.smartpowerbilling.com/api/chat/upload/file \
-H "Authorization: Bearer pk_live_xxx" \
-F "file=@screenshot-of-bill.pdf" -F "session_id=6f1c2a9e- ... "
```

RESPONSE 200

```
{ "status": "success", "filename": "screenshot-of-bill.pdf", "size": 182311,
  "path": "/tmp/uploads/acme_6f1c2a9e- ... _screenshot-of-bill.pdf", "message": "File uploaded
  successfully" }
```

Errors: **404** Session not found · **413** File size exceeds 10.0MB limit

POST `/api/chat/upload/image`

ANY KEY

Upload an image (multipart/form-data, field `image`). Same fields and limits as above.

FORM FIELDS (MULTIPART)

Field	Type	Req.	Description
<code>image</code>	file	yes	<code>image/jpeg</code> , <code>image/png</code> , <code>image/gif</code> or <code>image/webp</code> ; max 10 MB.

RESPONSE 200

```
{ "status": "success", "filename": "meter.jpg", "size": 90412, "path":
  "/tmp/uploads/acme_ ... _meter.jpg",
  "url": "/uploads/acme_ ... _meter.jpg", "message": "Image uploaded successfully" }
```

Errors: **400** File type not allowed · **413** too large

The returned `url` is informational; the server does not serve `/uploads/`.

Training

Server-side only: a secret key or a staff login (GET: viewer+, all other methods: trainer+). Every call that trains returns **202** with the new or updated **TrainingSource** and a `task_id`; the work runs in the background.

POST

/api/train/file

SECRET KEY / STAFF

Upload a document to train on (multipart/form-data).

FORM FIELDS (MULTIPART)

Field	Type	Req.	Description
file	file	yes	.pdf .docx .txt .md .csv .json .xml ; max 25 MB; not empty.
name	string	no	Display name (defaults to the file name).
audience	string	no	public customer vendor internal admin ; default internal .
tenant_id	string	no	As above.

EXAMPLE REQUEST

```
curl -X POST https://port.smartpowerbilling.com/api/train/file \
-H "Authorization: Bearer sk_live_xxx" \
-F "file=@Customer-Handbook.pdf" -F "name=Customer handbook" -F "audience=customer"
```

RESPONSE 202

```
{ "source": { "source_id": "3d0c6c1e-8f4b-4f57-9a55-2c1b6e7d9f10", "tenant_id": "acme",
  "name": "Customer handbook", "kind": "file", "source_type": "pdf",
  "filename": "Customer-Handbook.pdf", "url": null, "audience": "customer", "status": "queued",
  "documents_count": 0, "records_count": 0, "last_task_id": "a7e2 ... ", "last_synced_at": null,
  "error_message": null, "created_at": "2026-10-08T09:20:01+00:00", "updated_at": "2026-10-
08T09:20:01+00:00",
  "config": null },
  "task_id": "a7e2c9d4-5b1f-4a0e-8c3d-9f6b2e1a7c55" }
```

Errors: **400** The uploaded file has no filename / is empty · **413** File exceeds the 25 MB limit · **415** Unsupported file type · **422** bad audience · **403** publishable key or viewer

POST

/api/train/text

SECRET KEY / STAFF

RESPONSE 202

```
{ "source": { "source_id": "...", "name": "Refund policy", "kind": "text", "source_type": "text",
              "audience": "public", "status": "queued", ... },
  "task_id": "... " }
```

Errors: **422** content missing or blank, bad audience

POST /api/train/api-source/test

SECRET KEY / STAFF

Dry run: fetch the **first page**, find and normalize the records, return a preview. Nothing is stored. Body: the same **ApiSourceConfig** as the next endpoint.

EXAMPLE REQUEST

```
{
  "name": "Helpdesk tickets", "audience": "internal",
  "url": "https://help.example.com/api/tickets", "method": "GET",
  "headers": { "Authorization": "Bearer <helpdesk-read-only-token>" },
  "query": { "status": "solved" },
  "source_type": "support_tickets",
  "mapping": { "id_field": "ticket_id", "title_field": "subject",
               "question_field": "description", "messages_field": "messages" },
  "pagination": { "type": "cursor", "cursor_path": "meta.next_cursor", "cursor_param": "cursor",
                  "max_pages": 20 }
}
```

RESPONSE 200

```
{ "ok": true, "status_code": 200, "records_found": 2, "data_path": "data.tickets",
  "records_skipped": 0,
  "detected_fields": ["ticket_id", "subject", "description", "status", "priority", "messages"],
  "preview": [ { "title": "Support ticket #T-1042: Meter shows double usage",
                  "text": "Support ticket #T-1042: Meter shows double usage\nCustomer issue: My
meter ... " } ],
  "error": null }
```

Errors: **422** invalid config (e.g. url not http/https)

Remote failures never produce a 5xx: you get **ok: false** with **error** (e.g. "API returned HTTP 401: ...", "Request timed out after 30s", "API response is not valid JSON", "No records found in the response. Set data_path...", "Records were found but none had a recognizable issue/response. Set mapping.question_field / mapping.answer_field."). **preview** has at most 5 items.

POST /api/train/api-source

SECRET KEY / STAFF

Save an API source and train on all its pages (the first sync starts immediately).

BODY (JSON): APISOURCECONFIG

Field	Type	Req.	Description
url	string	yes	http:// or https:// address of the JSON API.
name	string	no	Display name (defaults to the URL's host).
audience	string	no	Default internal.
method	string	no	GET (default) or POST.
headers	object	no	String map, e.g. {"Authorization": "Bearer ..."} . Accept: application/json is added.

Field	Type	Req.	Description
query	object	no	String map of query parameters.
body	any	no	POST only. Objects/arrays are sent as JSON, anything else as raw text.
data_path	string	no	Dot path to the list of records (auto-detected when empty).
source_type	string	no	<code>support_tickets</code> (default) or <code>api_generic</code> .
mapping	object	no	<code>id_field</code> , <code>question_field</code> , <code>answer_field</code> , <code>title_field</code> , <code>messages_field</code> (dot paths), <code>extra_fields</code> (list). See section 3.
pagination	object	no	<code>type</code> : <code>none</code> <code>page</code> <code>cursor</code> (default <code>none</code>); <code>page_param</code> (default <code>page</code>); <code>cursor_path</code> (auto); <code>cursor_param</code> (default <code>cursor</code>); <code>max_pages</code> 1–100 (default 10).
tenant_id	string	no	Defaults to the credential's tenant; 403 if it differs.

EXAMPLE REQUEST

```
{
  "name": "Helpdesk tickets", "audience": "internal",
  "url": "https://help.example.com/api/tickets", "method": "GET",
  "headers": { "Authorization": "Bearer <helpdesk-read-only-token>" },
  "query": { "status": "solved" },
  "source_type": "support_tickets",
  "mapping": { "id_field": "ticket_id", "title_field": "subject",
    "question_field": "description", "messages_field": "messages" },
  "pagination": { "type": "cursor", "cursor_path": "meta.next_cursor", "cursor_param": "cursor",
    "max_pages": 20 }
}
```

RESPONSE 202

```
{ "source": { "source_id": "c51f...", "name": "Helpdesk tickets", "kind": "api",
  "source_type": "support_tickets", "url": "https://help.example.com/api/tickets",
  "audience": "internal", "status": "queued",
  "config": { "kind": "api", "method": "GET", "headers": { "Authorization": "...." },
  ... }, ... },
  "task_id": "... " }
```

Errors: **422** invalid config

The stored `config` (headers, query, body, data_path, source_type, mapping, pagination) is returned with secret-looking values replaced by `....`. It can't be edited later: delete and re-create the source to change it.

POST /api/train/sources/{source_id}/sync

SECRET KEY / STAFF

Fetch an API source again and update its knowledge (section 3, Re-sync).

QUERY PARAMETERS

Field	Type	Req.	Description
tenant_id	query	no	As above.

RESPONSE 202

```
{ "source": { ... , "status": "queued" }, "task_id": " ... " }
```

Errors: **400** Only API sources can be synced · **404** Training source not found · **409** This source is already syncing (unless stuck for over 30 minutes)

GET /api/train/sources

SECRET KEY / STAFF

Every training source of the tenant, newest first, with totals.

QUERY PARAMETERS

Field	Type	Req.	Description
tenant_id	query	no	As above.

RESPONSE 200

```
{ "sources": [ { "source_id": "3d0c ... ", "name": "Customer handbook", "kind": "file",  
  "source_type": "pdf",  
    "filename": "Customer-Handbook.pdf", "audience": "customer", "status":  
  "completed",  
    "documents_count": 214, "records_count": 0,  
    "error_message": "2 of 120 pages had no extractable text and were skipped",  
    "last_synced_at": "2026-10-08T09:21:37+00:00", ... } ],  
  "totals": { "sources": 12, "documents": 3810 } }
```

Sources created through /api/ingest are not listed (they are not training sources).

GET /api/train/sources/{source_id}

SECRET KEY / STAFF

One source. Poll it after training until `status` is `completed` or `failed`.

Response 200: TrainingSource (see the object reference at the end of this section)

Errors: **404** Training source not found

PATCH /api/train/sources/{source_id}

SECRET KEY / STAFF

Rename and/or relabel a source. A new audience is applied to every chunk at once and the answer cache is cleared.

BODY (JSON)

Field	Type	Req.	Description
audience	string	no	New audience.
name	string	no	New name (not blank).

EXAMPLE REQUEST

```
{ "audience": "public" }
```

Response 200: TrainingSource

Errors: **404** Training source not found · **422** bad audience / blank name

DELETE /api/train/sources/{source_id}

SECRET KEY / STAFF

Delete a source and all its chunks. Answers stop using it immediately.

RESPONSE 200

```
{ "deleted": true, "documents_deleted": 214 }
```

Errors: **404** Training source not found · **500** Failed to delete source

Ingest (low level)

The original raw-content endpoint, kept for log and code pipelines. For documents and text prefer `/api/train/*`: ingested content does not appear in the sources list and can't be relabelled or deleted through the API.

POST /api/ingest

SECRET KEY / STAFF

Queue raw content for chunking and embedding.

BODY (JSON)

Field	Type	Req.	Description
source_type	string	yes	Chooses the chunker: <code>codebase</code> , <code>logs</code> , <code>nginx_logs</code> , <code>docker_logs</code> , <code>postgres_logs</code> , <code>json</code> , <code>csv</code> , <code>text</code> , <code>txt</code> , <code>md</code> , <code>document</code> . Anything else is chunked as logs.
content	string	yes	The raw content.
file_path	string	no	Stored with the chunks and shown in <code>search_results</code> .
metadata	object	no	Free-form metadata.
audience	string	no	If omitted the chunks have no audience and count as <code>LEGACY_DOC_AUDIENCE</code> (public by default).
content_hash	string	no	Accepted but not used (duplicates are detected per tenant automatically).
tenant_id	string	no	Defaults to the credential's tenant; 403 if it differs.

EXAMPLE REQUEST

```
{ "source_type": "logs", "content": "2026-10-08 09:00:01 ERROR payment gateway timeout ... ",  
  "file_path": "/var/log/billing.log", "audience": "internal" }
```

RESPONSE 200

```
{ "status": "queued", "task_id": "e3b1 ... ", "message": "Ingestion task queued for processing" }
```

GET `/api/ingest/status/{task_id}`

SECRET KEY / STAFF

Status of any background task: ingest jobs and training jobs (`task_id` from a train call).

RESPONSE 200

```
{ "task_id": "a7e2c9d4- ... ", "status": "completed", "error_message": null }
```

Errors: **404** Task not found (unknown, or another tenant's)

`status`: `queued`, `processing`, `completed` or `failed`.

Staff authentication

No credentials needed. The login-type responses all have the same shape.

POST `/api/staff/login`

PUBLIC

Sign in with email and password.

BODY (JSON)

Field	Type	Req.	Description
<code>email</code>	string	yes	Case-insensitive.
<code>password</code>	string	yes	

EXAMPLE REQUEST

```
{ "email": "ada@acme.com", "password": "<password>" }
```

RESPONSE 200

```
{ "access_token": "<staff-jwt>", "token_type": "bearer", "expires_in": 43200,  
  "user": { "id": 7, "tenant_id": "acme", "email": "ada@acme.com", "full_name": "Ada Lovelace",  
            "role": "trainer", "is_active": true, "notify_email": true, "notify_in_app": true,  
            "last_login_at": "2026-10-08T09:30:00+00:00", "created_at": "2026-09-  
01T08:00:00+00:00",  
            "invited": false } }
```

Errors: **401** Invalid email or password (also for inactive or pending-invite accounts) · **429** Too many failed attempts (10 per 15 min per email + IP)

POST /api/staff/accept-invite

PUBLIC

Finish an invitation: set the password and sign in.

BODY (JSON)

Field	Type	Req.	Description
token	string	yes	From the invite link (#/accept-invite?token=...).
password	string	yes	8–256 characters, at most 72 bytes.
full_name	string	no	Max 120.

Response 200: Same as login.

Errors: **400** This link is invalid or has expired · **422** password too short/long

POST /api/staff/forgot-password

PUBLIC

Email a password-reset link (valid 1 hour) if the account exists. Always answers ok.

BODY (JSON)

Field	Type	Req.	Description
email	string	yes	

RESPONSE 200

```
{ "ok": true }
```

At most one email per user every 2 minutes; needs SMTP and an absolute dashboard URL.

POST /api/staff/reset-password

PUBLIC

Set a new password with the reset token and sign in.

BODY (JSON)

Field	Type	Req.	Description
token	string	yes	From the reset link.
password	string	yes	8+ characters.

Response 200: Same as login.

Errors: **400** This link is invalid or has expired · **422** weak password

Staff: own account

GET /api/staff/me

STAFF · VIEWER+

The signed-in staff user.

Response 200: StaffUser

PATCH /api/staff/me

STAFF · VIEWER+

Update your name and notification choices.

BODY (JSON)

Field	Type	Req.	Description
full_name	string	no	Max 120; blank clears it.
notify_email	boolean	no	Alert emails on/off.
notify_in_app	boolean	no	In-app alerts on/off.

EXAMPLE REQUEST

```
{ "notify_email": false }
```

Response 200: StaffUser

POST /api/staff/me/password

STAFF · VIEWER+

Change your password.

BODY (JSON)

Field	Type	Req.	Description
current_password	string	yes	
new_password	string	yes	8–256 characters, ≤72 bytes.

RESPONSE 200

```
{ "ok": true }
```

Errors: **400** Current password is incorrect · **422** weak password

Staff management

GET /api/staff

STAFF · VIEWER+

All staff of the tenant (including pending invitations), oldest first.

RESPONSE 200

```
{ "staff": [ StaffUser, ... ] }
```

POST /api/staff/invite

STAFF · ADMIN+

Invite a colleague. Admins can invite trainers and viewers; owners can invite any role.

BODY (JSON)

Field	Type	Req.	Description
email	string	yes	Must not exist anywhere yet.

Field	Type	Req.	Description
full_name	string	no	Max 120.
role	string	yes	owner admin trainer viewer.

EXAMPLE REQUEST

```
{ "email": "grace@acme.com", "full_name": "Grace Hopper", "role": "trainer" }
```

RESPONSE 200

```
{ "user": { ..., "email": "grace@acme.com", "role": "trainer", "invited": true },
  "invite_url": "https://dashboard.example.com/#/accept-invite?token=<invite-token>",
  "email_sent": true }
```

Errors: **403** Your role (admin) can't do this (role at or above your own) · **409** A staff member with this email already exists / This email is already registered · **422** Enter a valid email address / bad role

Without SMTP, `email_sent` is false: give the `invite_url` to the person yourself. It is valid 7 days.

PATCH `/api/staff/{user_id}`

STAFF · ADMIN+

Change someone's role, active flag, name or email alerts.

BODY (JSON)

Field	Type	Req.	Description
role	string	no	New role (must be below yours unless you are an owner).
is_active	boolean	no	false = deactivated (can't sign in).
full_name	string	no	
notify_email	boolean	no	

Response 200: StaffUser

Errors: **403** role too high · **404** Staff member not found · **409** A tenant must keep at least one active owner

DELETE `/api/staff/{user_id}`

STAFF · ADMIN+

Remove a staff member (and their notifications).

RESPONSE 200

```
{ "deleted": true }
```

Errors: **403** · **404** · **409** last active owner

POST /api/staff/{user_id}/resend-invite

STAFF · ADMIN+

New invite link (the old one stops working) and email.

RESPONSE 200

```
{ "invite_url": "https://dashboard.example.com/#/accept-invite?token=<invite-token>",
  "email_sent": true }
```

Errors: 400 This user has already accepted the invite

Knowledge gaps

GET /api/reports/gaps

STAFF · VIEWER+

The gap inbox, most recently asked first.

QUERY PARAMETERS

Field	Type	Req.	Description
status	string	no	open (default), resolved, dismissed or all.
reason	string	no	no_context low_confidence llm_unknown llm_unavailable negative_feedback.
q	string	no	Text search in the question (max 200).
page	integer	no	Default 1.
page_size	integer	no	Default 20, max 100.

RESPONSE 200

```
{ "items": [ KnowledgeGap, ... ], "total": 37, "page": 1, "page_size": 20,
  "counts": { "open": 37, "resolved": 112, "dismissed": 9 } }
```

Errors: 422 bad status or reason

counts ignore the status filter but respect reason and q (handy for tab badges).

GET /api/reports/gaps/{gap_id}

STAFF · VIEWER+

One gap.

Response 200: KnowledgeGap

Errors: 404 Knowledge gap not found

PATCH

`/api/reports/gaps/{gap_id}`

STAFF · TRAINER+

Resolve, dismiss or reopen a gap, or set its note.

BODY (JSON)

Field	Type	Req.	Description
status	string	no	open resolved dismissed.
resolution_note	string	no	Max 4,000; blank clears it.

EXAMPLE REQUEST

```
{ "status": "dismissed", "resolution_note": "Out of scope" }
```

Response 200: KnowledgeGap

Errors: 422 bad status

POST

`/api/reports/gaps/{gap_id}/answer`

STAFF · TRAINER+

Answer & train: creates a text training source from the answer and resolves the gap.

BODY (JSON)

Field	Type	Req.	Description
answer	string	yes	1–20,000 characters.
audience	string	no	Default public.
also_resolve_similar	boolean	no	Default false. Also resolves open gaps with ≥ 60% word overlap.

EXAMPLE REQUEST

```
{ "answer": "Estimated readings are corrected automatically on your next actual reading.",  
  "audience": "customer", "also_resolve_similar": true }
```

RESPONSE 200

```
{ "gap": { ... , "status": "resolved", "resolved_by": "Ada Lovelace", "source_id": "9b1e... " },  
  "source": { "source_id": "9b1e... ", "name": "Answer: Why was my reading estimated?", "kind":  
    "text",  
              "audience": "customer", "status": "queued", ... },  
  "task_id": "... ", "resolved_similar": 2 }
```

Errors: 404 Knowledge gap not found · 422 empty answer, bad audience

Notifications

Each staff user sees only their own notifications.

GET

/api/notifications

STAFF · VIEWER+

Newest first.

QUERY PARAMETERS

Field	Type	Req.	Description
unread_only	boolean	no	Default false.
limit	integer	no	1–100, default 30.

RESPONSE 200

```
{ "items": [ { "id": 88, "type": "gap", "title": "New unanswered question",  
  "body": "Why was my reading estimated? (no matching knowledge; asked by Ada)",  
  "link": "#/gaps/41", "read": false, "created_at": "2026-10-08T09:40:12+00:00" } ],  
  "unread_count": 3 }
```

type: gap, feedback, training_failed or staff. link is a dashboard hash route.

POST

/api/notifications/{notification_id}/read

STAFF · VIEWER+

Mark one as read.

RESPONSE 200

```
{ "ok": true }
```

Errors: 404 Notification not found

POST

/api/notifications/read-all

STAFF · VIEWER+

Mark all as read.

RESPONSE 200

```
{ "ok": true, "updated": 3 }
```

Reports

GET

/api/reports/overview

STAFF · VIEWER+

Dashboard analytics for the last days days (including today, UTC).

QUERY PARAMETERS

Field	Type	Req.	Description
days	integer	no	1–365, default 30.

RESPONSE 200

```
{ "range": { "from": "2026-09-09", "to": "2026-10-08", "days": 30 },
  "totals": { "conversations": 412, "questions": 1290, "answered": 1187, "unanswered": 103,
    "answer_rate": 0.9202, "gaps_open": 37, "sources": 12, "documents": 3810, "staff": 6
  },
  "daily": [ { "date": "2026-09-09", "questions": 38, "unanswered": 4 }, ... ],
  "by_role": [ { "role": "public", "questions": 900 }, { "role": "customer", "questions": 390 } ],
  "top_questions": [ { "question": "How do I pay my bill?", "count": 57 }, ... ],
  "recent_gaps": [ KnowledgeGap, ... ],
  "sources_by_status": { "completed": 11, "processing": 0, "queued": 0, "failed": 1 } }
```

questions = user messages in range; unanswered = gap events in range (cached weak answers included); answered = questions – unanswered; top_questions: 10 most frequent; recent_gaps: 5 latest open gaps.

GET /api/reports/conversations

STAFF · VIEWER+

All conversations of the tenant, most recently active first.

QUERY PARAMETERS

Field	Type	Req.	Description
q	string	no	Search in title, user name/id, session id and message text.
user_role	string	no	Filter by role (public includes sessions without a role).
page	integer	no	Default 1.
page_size	integer	no	Default 20, max 100.

RESPONSE 200

```
{ "items": [ { "session_id": "6f1c ... ", "title": "Why is my bill twice as high this month?",
  "user_id": "shop:42", "user_name": "Ada", "user_role": "public", "message_count":
4,
    "created_at": " ... ", "last_message_at": " ... ", "has_gap": true } ],
  "total": 412, "page": 1, "page_size": 20 }
```

GET /api/reports/conversations/{session_id}

STAFF · VIEWER+

One conversation with gap reasons and user ratings per answer.

RESPONSE 200

```
{ "session_id": "6f1c ... ", "title": " ... ", "user_id": "shop:42", "user_name": "Ada", "user_role":
"public",
  "created_at": " ... ",
  "messages": [ { "id": 5121, "role": "assistant", "content": " ... ", "created_at": " ... ",
    "gap_reason": "low_confidence", "feedback": "down" } ] }
```

Errors: 404 Conversation not found

Settings and widget configuration

GET

/api/settings

STAFF · VIEWER+

The tenant's settings, with defaults filled in.

Response 200: TenantSettings

PUT

/api/settings

STAFF · ADMIN+

Partial update: send only the keys to change; `widget` is merged key by key. Unknown keys are ignored; `tenant_id` and `smtp_configured` are read-only.

BODY (JSON)

Field	Type	Req.	Description
display_name	string null	no	Max 120 (used in emails).
support_email	string null	no	Valid email.
gap_distance_threshold	number	no	0–2 (default 0.55).
notify_on_gap	boolean	no	Email on new gaps.
notify_on_feedback	boolean	no	Email on thumbs-down.
email_max_per_hour	integer	no	0–1000 (default 20).
widget	object	no	<code>theme</code> { <code>primary</code> , <code>panel</code> , <code>ink</code> } (64 chars each, not empty); <code>branding</code> { <code>botName</code> ≤80, <code>title</code> ≤120, <code>subtitle</code> ≤240, <code>logoUrl</code> ≤2048, <code>http(s)/data:image/relative</code> }; <code>launcher</code> { <code>type</code> ≤20, <code>text</code> ≤60}; <code>welcomeMessages</code> ≤10 × 500; <code>quickActions</code> ≤12 × 120; <code>placeholder</code> ≤200. Max 32 KB.

EXAMPLE REQUEST

```
{ "gap_distance_threshold": 0.5, "email_max_per_hour": 10,
  "widget": { "theme": { "primary": "#1d4ed8" }, "branding": { "botName": "Acme Assistant" },
    "welcomeMessages": ["Hi! Ask me about your bill."] } }
```

Response 200: TenantSettings

Errors: **422** validation message, e.g. `gap_distance_threshold` must be a number between 0 and 2

GET

/api/widget-config

ANY KEY

The public chat-widget appearance, so a website can style its widget from the dashboard settings.

QUERY PARAMETERS

Field	Type	Req.	Description
tenant_id	query	no	As above.

RESPONSE 200

```
{ "widget": { "theme": { "primary": "#b93fff", "panel": "#fff8ff", "ink": "#08080a" },
  "branding": { "botName": "Brainbox", "title": null, "subtitle": null, "logoUrl":
null },
  "launcher": { "type": "button", "text": "Chat" },
  "welcomeMessages": ["Hi! How can I help you today?"], "quickActions": [],
  "placeholder": "Type message ..." } }
```

The values shown are the defaults. Note: the sdk-web widget does not fetch this automatically yet; pass the values to `Brainbox.init()`.

API keys (dashboard)

Keys of the signed-in user's own tenant.

GET `/api/keys`

STAFF · ADMIN+

All keys of the tenant (never the raw keys), newest first.

RESPONSE 200

```
{ "keys": [ { "id": 12, "name": "Website widget", "key_type": "publishable", "key_prefix":
"pk_live_Ab3x",
  "is_active": true, "created_at": "...", "last_used": "...", "expires_at": null,
  "expired": false } ] }
```

POST `/api/keys`

STAFF · ADMIN+

Create a key. The raw key is returned **once**.

BODY (JSON)

Field	Type	Req.	Description
name	string	yes	1–120 characters.
key_type	string	yes	publishable or secret.
expires_at	datetime	no	ISO date-time; expired keys get 401.

EXAMPLE REQUEST

```
{ "name": "Odooserver", "key_type": "secret", "expires_at": "2027-12-31T23:59:59Z" }
```

RESPONSE 201

```
{ "key": { "id": 13, "name": "Odooserver", "key_type": "secret", "key_prefix": "sk_live_9QzT",
... },
  "raw_key": "sk_live_xxx" }
```

Errors: **422** invalid

DELETE**/api/keys/{key_id}****STAFF · ADMIN+**

Revoke a key (immediately stops working).

RESPONSE 200

```
{ "revoked": true }
```

Errors: **404** API key not found

Admin (server operator)

Header **X-Admin-Token:** <admin-token> (value of **BRAINBOX_ADMIN_TOKEN**). Works across all tenants. **503** when the token isn't configured on the server, **401** when the header is missing or wrong.

POST**/api/admin/keys****ADMIN TOKEN**

Create a key for any tenant, or import an existing raw key so already-deployed sites keep working.

BODY (JSON)

Field	Type	Req.	Description
tenant_id	string	yes	Target tenant.
key_type	string	yes	publishable secret.
name	string	no	Default "<type> key for <tenant>".
expires_at	datetime	no	
key	string	no	Import this raw key (≥ 24 chars, no spaces; a pk/sk_ prefix must match key_type).

EXAMPLE REQUEST

```
{ "tenant_id": "acme", "key_type": "publishable", "name": "Website widget" }
```

RESPONSE 201

```
{ "id": 12, "tenant_id": "acme", "name": "Website widget", "key_type": "publishable",  
  "key_prefix": "pk_live_Ab3x", "is_active": true, "expired": false, "created_at": "...",  
  "expires_at": null, "last_used": null, "api_key": "pk_live_xxx" }
```

Errors: **409** This key already exists · **422** invalid

GET

/api/admin/keys

ADMIN TOKEN

List keys, optionally for one tenant.

QUERY PARAMETERS

Field	Type	Req.	Description
tenant_id	query	no	Filter by tenant.

RESPONSE 200

```
{ "keys": [ { "id": 12, "tenant_id": "acme", ... } ] }
```

DELETE

/api/admin/keys/{key_id}

ADMIN TOKEN

Revoke a key.

Response 200: The revoked key object (`is_active: false`).

Errors: **404** API key not found

POST

/api/admin/staff

ADMIN TOKEN

Create a staff user in any tenant, typically a new company's first owner.

BODY (JSON)

Field	Type	Req.	Description
tenant_id	string	yes	
email	string	yes	
full_name	string	no	
role	string	no	Default <code>owner</code> .
password	string	no	Omit to get an invite link instead.

EXAMPLE REQUEST

```
{ "tenant_id": "acme", "email": "owner@acme.com", "full_name": "Ada Lovelace", "role": "owner" }
```

RESPONSE 201

```
{ "user": StaffUser, "invite_url": "https://dashboard.example.com/#/accept-invite?token=<invite-token>" }
```

Errors: **409** email exists · **422** invalid email, role or password

Legacy: /api/auth

Kept for compatibility with early clients. Not used by the dashboard; new integrations should not use them.

POST /api/auth/signup

PUBLIC

Creates a legacy user account (not a staff account).

BODY (JSON)

Field	Type	Req.	Description
username	string	yes	
password	string	yes	
email	string	yes	

RESPONSE 200

```
{ "user_id": 3, "username": "demo", "email": "demo@example.com", "message": "User created successfully" }
```

Errors: **400** Username already exists

Open to anyone. The account has no tenant or role and can't access anything else.

POST /api/auth/login

PUBLIC

Legacy login.

BODY (JSON)

Field	Type	Req.	Description
username	string	yes	
password	string	yes	

RESPONSE 200

```
{ "access_token": "<legacy-jwt>", "token_type": "bearer" }
```

Errors: **401** Invalid credentials · **403** User is not active

This token is **not** a staff login: other endpoints reject it (401 "Invalid or expired session").

POST /api/auth/api-key

ADMIN TOKEN

Deprecated key creation; use `POST /api/admin/keys`.

QUERY PARAMETERS

Field	Type	Req.	Description
tenant_id	query	yes	
key_type	query	no	publishable (default) or secret.

RESPONSE 200

```
{ "api_key": "pk_live_xxx", "tenant_id": "acme", "key_type": "publishable" }
```

Object reference

Objects returned by several endpoints above.

TrainingSource

Field	Type	Description
source_id	string (UUID)	Id used in /api/train/sources/{source_id}.
tenant_id	string	
name	string	Display name.
kind	string	file, text or api.
source_type	string	File extension (pdf, docx ...), text, support_tickets or api_generic.
filename / url	string null	Original file name (files) or API address (API sources).
audience	string	Who may see its chunks.
status	string	queued → processing → completed failed.
documents_count	integer	Chunks currently stored.
records_count	integer	Tickets/records found (structured sources).
last_task_id	string null	Latest background task.
last_synced_at	datetime null	Last successful run.
error_message	string null	Failure reason, or notes on a completed run (skipped pages/records).
created_at / updated_at	datetime	
config	object null	API sources only, secrets shown as

KnowledgeGap

Field	Type	Description
id	integer	

Field	Type	Description
<code>tenant_id</code>	string	
<code>question</code>	string	As first asked.
<code>reason</code>	string	<code>no_context</code> <code>low_confidence</code> <code>llm_unknown</code> <code>llm_unavailable</code> <code>negative_feedback</code> .
<code>status</code>	string	<code>open</code> <code>resolved</code> <code>dismissed</code> .
<code>occurrences</code>	integer	Times asked.
<code>best_distance</code>	number null	Closest match distance.
<code>answer_given</code>	string null	What Brainbox answered.
<code>session_id, user_id, user_name, user_role</code>	string null	Latest asker.
<code>resolution_note</code>	string null	
<code>resolved_by</code>	string null	Staff name.
<code>resolved_at</code>	datetime null	
<code>source_id</code>	string null	Training source created by Answer & train.
<code>created_at, last_seen_at</code>	datetime	
<code>last_feedback_comment</code>	string null	Latest thumbs-down comment (additive).

StaffUser

Field	Type	Description
<code>id</code>	integer	
<code>tenant_id</code>	string	
<code>email</code>	string	Lower-case.
<code>full_name</code>	string null	
<code>role</code>	string	<code>owner</code> <code>admin</code> <code>trainer</code> <code>viewer</code> .
<code>is_active</code>	boolean	
<code>notify_email, notify_in_app</code>	boolean	Alert preferences.
<code>last_login_at, created_at</code>	datetime null	
<code>invited</code>	boolean	true while the invitation hasn't been accepted.

TenantSettings

Field	Type	Description
<code>tenant_id</code>	string	Read-only.
<code>display_name</code>	string	Defaults to the tenant id.
<code>support_email</code>	string null	
<code>gap_distance_threshold</code>	number	Default 0.55 (server setting GAP_DISTANCE_THRESHOLD).

Field	Type	Description
<code>notify_on_gap, notify_on_feedback</code>	boolean	Default true.
<code>email_max_per_hour</code>	integer	Default 20.
<code>smtp_configured</code>	boolean	Read-only: can the server send email?
<code>widget</code>	object	See <code>/api/widget-config</code> .

11 Quick-start recipes

Copy-paste examples for the common jobs, each in curl, JavaScript (`fetch`) and Python (`requests`). Training examples run on a **server** with a secret key; recipe 6 is the only one meant for a browser.

Setup used by every recipe

CURL (BASH)

```
export BB=https://port.smartpowerbilling.com
export SK=sk_live_xxx          # secret key: servers and scripts only
```

JAVASCRIPT (NODE 18+, SERVER SIDE)

```
const BB = 'https://port.smartpowerbilling.com';
const SK = process.env.BRAINBOX_SECRET_KEY;          // sk_live_xxx, never ship it to a browser

async function bb(method, path, body, token = SK) {
  const isForm = body instanceof FormData;
  const res = await fetch(BB + path, {
    method,
    headers: { Authorization: `Bearer ${token}`, ...(body && !isForm ? { 'Content-Type':
'application/json' } : {}) },
    body: isForm ? body : body ? JSON.stringify(body) : undefined,
    signal: AbortSignal.timeout(200_000),            // chat can take ~180 s
  });
  const data = await res.json().catch(() => ({}));
  if (!res.ok) throw new Error(`${res.status} ${JSON.stringify(data.detail)}`);
  return data;
}
```

PYTHON 3 (REQUESTS)

```
import os, time, requests

BB = "https://port.smartpowerbilling.com"
SK = os.environ["BRAINBOX_SECRET_KEY"]              # sk_live_xxx

def bb(method, path, token=SK, **kw):
    r = requests.request(method, BB + path, headers={"Authorization": f"Bearer {token}"},
                        timeout=kw.pop("timeout", 200), **kw)
    if not r.ok:
        raise RuntimeError(f"{r.status_code} {r.text}")
    return r.json()
```

1 • Train a PDF

CURL

```
curl -X POST "$BB/api/train/file" -H "Authorization: Bearer $SK" \
  -F "file=@Customer-Handbook.pdf" -F "name=Customer handbook" -F "audience=customer"
```

JAVASCRIPT

```
import { readFile } from 'node:fs/promises';
const form = new FormData();
form.append('file', new Blob([await readFile('Customer-Handbook.pdf')], { type: 'application/pdf' })),
'Customer-Handbook.pdf');
form.append('name', 'Customer handbook');
form.append('audience', 'customer');
const { source, task_id } = await bb('POST', '/api/train/file', form);
```

PYTHON

```
with open("Customer-Handbook.pdf", "rb") as f:
    res = bb("POST", "/api/train/file", files={"file": ("Customer-Handbook.pdf", f,
"application/pdf")},
            data={"name": "Customer handbook", "audience": "customer"})
    source_id = res["source"]["source_id"]
```

2 · Train pasted text

CURL

```
curl -X POST "$BB/api/train/text" -H "Authorization: Bearer $SK" -H "Content-Type: application/json" \
-d '{"name": "Refund policy", "audience": "public", "content": "Refunds are paid back to the original
card within 5 working days of approval."}'
```

JAVASCRIPT

```
const { source } = await bb('POST', '/api/train/text', {
  name: 'Refund policy', audience: 'public',
  content: 'Refunds are paid back to the original card within 5 working days of approval.',
});
```

PYTHON

```
res = bb("POST", "/api/train/text", json={
    "name": "Refund policy", "audience": "public",
    "content": "Refunds are paid back to the original card within 5 working days of approval."})
```

3 · Connect a support system: test, then save

CURL

```
cat > helpdesk.json <<'EOF'
{ "name": "Helpdesk tickets", "audience": "internal",
  "url": "https://help.example.com/api/tickets",
  "headers": { "Authorization": "Bearer <helpdesk-read-only-token>" },
  "source_type": "support_tickets",
  "mapping": { "id_field": "ticket_id", "question_field": "description", "messages_field": "messages"
},
  "pagination": { "type": "cursor", "max_pages": 20 } }
EOF
curl -X POST "$BB/api/train/api-source/test" -H "Authorization: Bearer $SK" \
-H "Content-Type: application/json" -d @helpdesk.json      # check "ok" and "preview"
curl -X POST "$BB/api/train/api-source" -H "Authorization: Bearer $SK" \
-H "Content-Type: application/json" -d @helpdesk.json      # saves and starts the first sync
```

JAVASCRIPT

```
const config = {
  name: 'Helpdesk tickets', audience: 'internal',
  url: 'https://help.example.com/api/tickets',
  headers: { Authorization: `Bearer ${process.env.HELPDESK_TOKEN}` },
  source_type: 'support_tickets',
  mapping: { id_field: 'ticket_id', question_field: 'description', messages_field: 'messages' },
  pagination: { type: 'cursor', max_pages: 20 },
};
const test = await bb('POST', '/api/train/api-source/test', config);
if (!test.ok) throw new Error(test.error); // e.g. set data_path or mapping
console.log(test.records_found, test.detected_fields, test.preview[0]?.text);
const { source } = await bb('POST', '/api/train/api-source', config);
```

PYTHON

```
config = {
    "name": "Helpdesk tickets", "audience": "internal",
    "url": "https://help.example.com/api/tickets",
    "headers": {"Authorization": f"Bearer {os.environ['HELPDESK_TOKEN']}"},
    "source_type": "support_tickets",
    "mapping": {"id_field": "ticket_id", "question_field": "description", "messages_field":
"messages"},
    "pagination": {"type": "cursor", "max_pages": 20},
}
test = bb("POST", "/api/train/api-source/test", json=config)
assert test["ok"], test["error"]
source = bb("POST", "/api/train/api-source", json=config)["source"]
# later, e.g. nightly from cron: bb("POST", f"/api/train/sources/{source['source_id']}/sync")
```

4 · List sources and wait for training to finish

CURL

```
curl "$BB/api/train/sources" -H "Authorization: Bearer $SK"
curl "$BB/api/train/sources/3d0c6c1e-8f4b-4f57-9a55-2c1b6e7d9f10" -H "Authorization: Bearer $SK" #
one source
```

JAVASCRIPT

```
async function waitForSource(id) {
  for (;;) {
    const s = await bb('GET', `/api/train/sources/${id}`);
    if (s.status === 'completed' || s.status === 'failed') return s; // s.error_message explains
    await new Promise((r) => setTimeout(r, 3000));
  }
}
const { sources, totals } = await bb('GET', '/api/train/sources');
```

PYTHON

```
def wait_for_source(source_id):
    while True:
        s = bb("GET", f"/api/train/sources/{source_id}")
        if s["status"] in ("completed", "failed"):
            return s # s["documents_count"], s["error_message"]
        time.sleep(3)
```

5 · Change who can see a source

CURL

```
curl -X PATCH "$BB/api/train/sources/3d0c6c1e-8f4b-4f57-9a55-2c1b6e7d9f10" \
-H "Authorization: Bearer $SK" -H "Content-Type: application/json" -d '{"audience": "internal"}'
```

JAVASCRIPT

```
await bb('PATCH', `/api/train/sources/${sourceId}`, { audience: 'internal' });
```

PYTHON

```
bb("PATCH", f"/api/train/sources/{source_id}", json={"audience": "internal"})
```

6 · Chat from a website with a publishable key

The publishable key may sit in the page. Every visitor is `public`, so only public knowledge is used. Keep the `session_id` to continue the conversation and the `message_id` for feedback.

JAVASCRIPT (BROWSER)

```
const BB = 'https://port.smartpowerbilling.com';
const PK = 'pk_live_xxx'; // publishable: safe in a page
let sessionId = localStorage.getItem('bb-session');

async function ask(question) {
  const res = await fetch(`${BB}/api/chat`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json', Authorization: `Bearer ${PK}` },
    body: JSON.stringify({ question, session_id: sessionId || undefined }),
    signal: AbortSignal.timeout(200_000),
  });
  if (!res.ok) throw new Error((await res.json()).detail);
  const data = await res.json();
  sessionId = data.session_id; localStorage.setItem('bb-session', sessionId);
  return data; // data.response, data.message_id
}
```

CURL

```
curl -X POST "$BB/api/chat" -H "Authorization: Bearer pk_live_xxx" -H "Content-Type: application/json" \
--max-time 200 -d '{"question": "How do I pay my bill?"}'
```

PYTHON (SERVER SIDE, WITH A TRUSTED ROLE)

```
# From your own backend, after you've checked who the user is:
ans = bb("POST", "/api/chat", json={"question": "Which tariffs can I switch to?",
                                     "user_id": "shop:42", "user_name": "Ada", "user_role":
                                     "customer"})
print(ans["response"])
```


7 · Send feedback on an answer

CURL

```
curl -X POST "$BB/api/chat/feedback" -H "Authorization: Bearer pk_live_xxx" -H "Content-Type: application/json" \
  -d '{"session_id":"6f1c2a9e-1b7d-4a53-9d0e-2b8f7c41e0aa","message_id":5121,"rating":"down","comment":"Not my question"}'
```

JAVASCRIPT (BROWSER)

```
await fetch(`${BB}/api/chat/feedback`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json', Authorization: `Bearer ${PK}` },
  body: JSON.stringify({ session_id: sessionId, message_id: data.message_id, rating: 'down', comment: 'Not my question' }),
});
```

PYTHON

```
bb("POST", "/api/chat/feedback", json={"session_id": ans["session_id"], "message_id": ans["message_id"],
                                     "rating": "up"})
```

8 · List open knowledge gaps and answer one (staff login)

CURL

```
TOKEN=$(curl -s -X POST "$BB/api/staff/login" -H "Content-Type: application/json" \
  -d '{"email":"ada@acme.com","password":"<password>"}' | python -c "import sys,json;print(json.load(sys.stdin)['access_token'])")
curl "$BB/api/reports/gaps?status=open&page_size=50" -H "Authorization: Bearer $TOKEN"
curl -X POST "$BB/api/reports/gaps/41/answer" -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"answer":"Estimated readings are corrected on your next actual reading.", "audience":"customer", "also_resolve_similar":true}'
```

JAVASCRIPT

```
const login = await fetch(`${BB}/api/staff/login`, { method: 'POST', headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ email: 'ada@acme.com', password: process.env.STAFF_PASSWORD }) }).then((r) => r.json());
const jwt = login.access_token; // <staff-jwt>, valid 12 h
const { items, counts } = await bb('GET', '/api/reports/gaps?status=open&page_size=50', null, jwt);
await bb('POST', `/api/reports/gaps/${items[0].id}/answer`,
  { answer: 'Estimated readings are corrected on your next actual reading.', audience: 'customer', also_resolve_similar: true }, jwt);
```

PYTHON

```
jwt = requests.post(f"{BB}/api/staff/login", json={"email": "ada@acme.com",
                                                  "password": os.environ["STAFF_PASSWORD"]}, timeout=30).json()["access_token"]
gaps = bb("GET", "/api/reports/gaps", token=jwt, params={"status": "open", "reason": "no_context"})
for g in gaps["items"][:1]:
    bb("POST", f"/api/reports/gaps/{g['id']}/answer", token=jwt,
        json={"answer": "Estimated readings are corrected on your next actual reading.", "audience": "customer"})
```

9 · Invite a staff member (admin or owner)

CURL

```
curl -X POST "$BB/api/staff/invite" -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"email":"grace@acme.com","full_name":"Grace Hopper","role":"trainer"}'
```

JAVASCRIPT

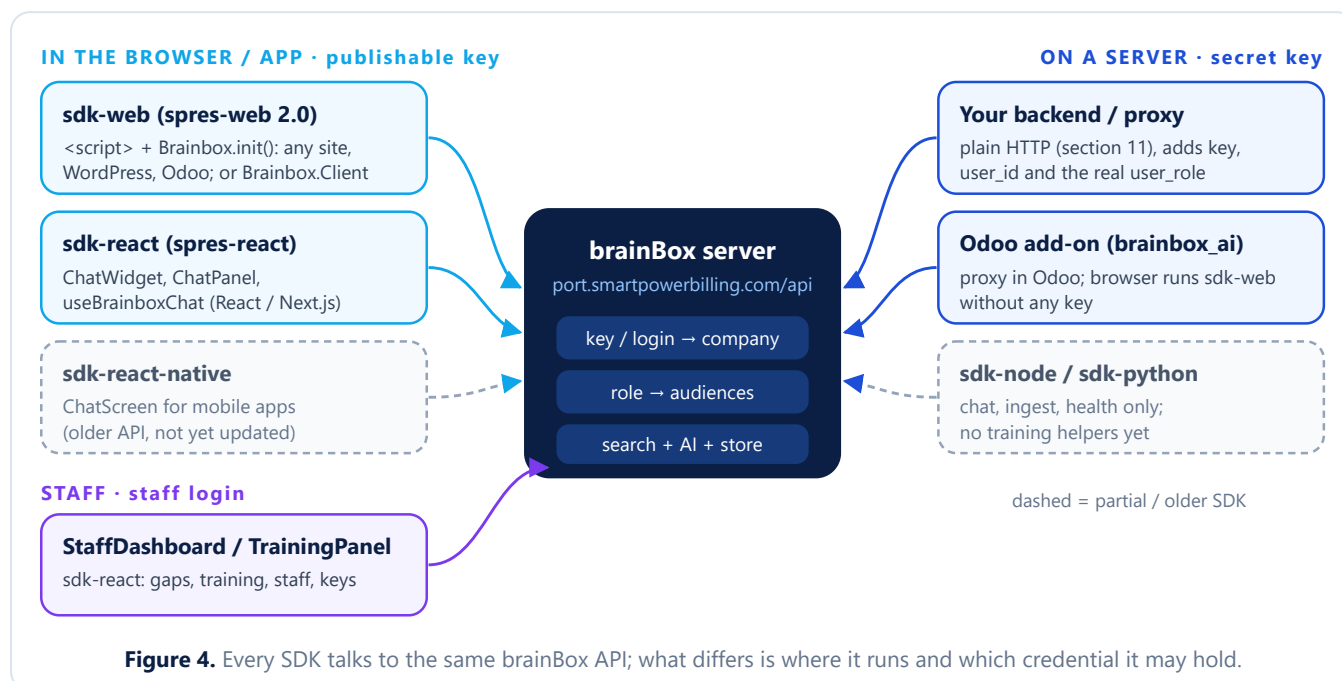
```
const { invite_url, email_sent } = await bb('POST', '/api/staff/invite',
  { email: 'grace@acme.com', full_name: 'Grace Hopper', role: 'trainer' }, jwt);
if (!email_sent) console.log('Send this link yourself (valid 7 days):', invite_url);
```

PYTHON

```
inv = bb("POST", "/api/staff/invite", token=jwt,
  json={"email": "grace@acme.com", "full_name": "Grace Hopper", "role": "trainer"})
print(inv["email_sent"], inv["invite_url"])
```

12 Using the SDKs

The SDKs are ready-made "faces" and helpers around the API above. They contain no AI; everything goes to the same brainBox server. Pick one by where your code runs.



SDK	Package	Runs in	What it covers today
sdk-web	spres-web 2.0.0	Browser, Node 18+	Chat widget (floating, sidebar, inline, full page), history, uploads, proxy mode, headless <code>Brainbox.Client</code> . No training helpers (only v1 <code>ingest</code>); no feedback buttons.
sdk-react	spres-react 1.0.3	React / Next.js	Chat UI, all training methods + <code>TrainingPanel</code> , staff dashboard + <code>BrainboxStaffClient</code> . No feedback helper on the chat client.
sdk-react-native	brainbox-react-native-sdk 1.0.0	React Native	Older client: chat, create session, ingest, health. Not yet updated for the current API (see below).
sdk-node	spres-ai 1.0.0	Node	<code>chat</code> , <code>createChatSession</code> , <code>ingest</code> , <code>getIngestStatus</code> , <code>healthCheck</code> (+ code-search helpers).
sdk-python	spres-ai 1.0.0	Python	<code>chat</code> , <code>create_chat_session</code> , <code>ingest</code> , <code>get_ingest_status</code> , <code>health_check</code> (+ code-search helpers).

sdk-web: any HTML page, WordPress, Odoo

One dependency-free JavaScript file. Host `brainbox-web-sdk.js` on your site (or use the npm package `spres-web`) and start the widget:

```
<script src="/static/brainbox-web-sdk.js"></script>
<script>
  const bb = Brainbox.init({
    apiUrl: 'https://port.smartpowerbilling.com', // without /api
    apiKey: 'pk_live_xxx', // publishable key only
    user: { id: 'shop:42', name: 'Ada' }, // optional: per-user history
    mode: 'floating', // floating | sidebar | inline | page
    branding: { botName: 'Acme Assistant', subtitle: 'Ask us anything' },
    theme: { primary: '#1d4ed8' }
  });
</script>
```

Useful options: `container` (for inline/page), `welcomeMessages`, `quickActions`, `context` (a function returning text prepended to each question, e.g. the record being viewed), `timeout` (default 200,000 ms), `onEvent`. Methods: `open()`, `close()`, `send(text)`, `newChat()`, `loadSession(id)`, `updateConfig()`, `destroy()`.

Proxy mode (recommended in production, and how the Odoo add-on works): point `apiUrl` at an endpoint on your own domain, leave out `apiKey`, `tenantId` and `user.id`. Your server checks the logged-in user, adds `Authorization: Bearer sk_live_xxx`, the real `user_id` and `user_role`, and forwards to brainBox. Users then get answers for their role and nobody can read another user's history.

```
Brainbox.init({ apiUrl: '/brainbox_ai/proxy', credentials: 'same-origin',
  headers: { 'X-CSRF-Token': csrfToken }, user: { name: currentUser.name } });
```

Headless: `new Brainbox.Client({ apiUrl, apiKey })` gives `chat()`, `listSessions()`, `getSessionMessages()`, `createSession()`, `uploadFile()`, `uploadImage()`, `health()` and a low-level `request(method, path, {body})` you can use for any endpoint, for example feedback: `client.request('POST', '/api/chat/feedback', { body: { ... } })`.

sdk-react: React and Next.js

```
import { BrainboxReactSDK, ChatWidget, ChatPanel, TrainingPanel, StaffDashboard } from 'spres-react';

// Public website chat: publishable key
const chatSdk = new BrainboxReactSDK('https://port.smartpowerbilling.com', 'pk_live_xxx');
export const SiteChat = () => <ChatWidget sdk={chatSdk} buttonText="Support" />;

// Internal admin tool behind your own login: secret key (server-rendered or internal only)
const trainSdk = new BrainboxReactSDK('https://port.smartpowerbilling.com', () => getInternalToken());
export const Train = () => <TrainingPanel sdk={trainSdk} companyName="Acme" />;

// Complete staff console: staff sign in with email + password, no key embedded
export const Staff = () => <StaffDashboard apiUrl="https://port.smartpowerbilling.com"
brandName="Acme" />;
```

- **Exports:** `BrainboxReactSDK`, `ChatWidget`, `ChatPanel`, `useBrainboxChat`, `TrainingPanel`, `useBrainboxTraining`, `StaffDashboard`, `BrainboxStaffClient`, `MessageContent`, `TypingIndicator`, plus types.
- **Training methods** on `BrainboxReactSDK`: `trainFile(file, {name, audience, onUploadProgress})`, `trainText(name, content, audience)`, `testApiSource(config)`, `addApiSource(config)`, `syncSource(id)`, `listSources()`, `getSource(id)`, `updateSource(id, {audience, name})`, `deleteSource(id)`, `getIngestStatus(taskId)`. Errors are `BrainboxApiError` with `status`, `detail` and `code` (`auth`, `scope`, `tenant`).

- The second constructor argument can be a **function** returning the current token (e.g. a staff JWT); `new BrainboxStaffClient({apiUrl}).sdk` is a ready training client signed in as the staff user.
- `StaffDashboard` routes: `#/overview`, `#/gaps`, `#/conversations`, `#/training`, `#/staff`, `#/settings`, `#/notifications`, `#/account`, plus `#/accept-invite` and `#/reset-password` for email links. Host it at `DASHBOARD_URL`.

STATUS

The staff dashboard is being finished: the pages and their API calls are in place, but expect visual polish and small changes before it is published. `streamChat()` does not stream from the server (there is no streaming endpoint); it gets the whole answer and reveals it word by word in the browser.

sdk-react-native

`BrainboxReactNativeSDK(apiUrl, apiKey, tenantId)` and a `ChatScreen` component. **Status: not yet updated** for the current API. It always sends `tenant_id` (pass the key's own tenant or `undefined`, otherwise 403), it has no session list, user scoping, uploads or feedback, and its `streamChat()` calls `/api/chat/stream`, which doesn't exist (use `chat()`). Use a publishable key in apps.

sdk-node and sdk-python (server side)

Both are thin clients with a secret key: `chat`, `create session`, `ingest` (raw content, low level), `ingest status` and `health`. Honest gaps today:

- **No training helpers** (file, text, API sources, list / relabel / delete sources), no gap, staff or report calls. Use plain HTTP for those (section 11): it is a few lines.
- `chat()` can't pass `user_id`, `user_name` or `user_role`, so answers are limited to **public** knowledge and sessions aren't per user.
- `ingest()` sends no `audience`: content becomes legacy (treated as `LEGACY_DOC_AUDIENCE`, public by default). Prefer `/api/train/text`.
- The Python client always sends `tenant_id`: pass the key's own tenant (or `None`).

```
from brainbox_sdk import BrainboxPythonSDK # pip package spres-ai
sdk = BrainboxPythonSDK("https://port.smartpowerbilling.com", "sk_live_xxx", None)
print(sdk.chat("What are your opening hours?")["response"])
```

13 Server admin appendix

For whoever runs the brainBox server. Settings live in `brainBox/.env` (start from `.env.example`); restart the server after changing them.

Environment variables

Variable	Example / default	Purpose
INFRASTRUCTURE		
DATABASE_URL	<code>postgresql://.../brainbox</code>	PostgreSQL with the pgvector extension (created on startup). Always set it ; don't rely on built-in defaults.
REDIS_URL	<code>redis://localhost:6379/0</code>	Answer cache.
CELERY_BROKER_URL, CELERY_RESULT_BACKEND	<code>redis://localhost:6379/0</code>	Optional worker queue (training uses in-process background tasks).
OLLAMA_BASE_URL, OLLAMA_MODEL	<code>http://localhost:11434 , llama2</code>	The AI model that writes answers (180 s timeout).
USE_OPENAI, OPENAI_API_KEY	<code>false</code> , empty	OpenAI as a fallback when Ollama gives no answer (both must be set).
EMBEDDING_MODEL	<code>BAAI/bge-base-en-v1.5</code>	Fingerprint model (768 dimensions). Changing it requires re-training everything.
DEBUG, LOG_LEVEL	<code>false</code> , <code>INFO</code>	Logging.
HOST, PORT	<code>0.0.0.0</code> , <code>8000</code>	Listen address. Not read by the app itself: pass them to your launcher (e.g. <code>uvicorn app.main:app --host 0.0.0.0 --port 8000</code>).
SECURITY AND KEYS		
REQUIRE_API_KEY	<code>true</code>	Enforce API keys everywhere except <code>GET /</code> and <code>GET /api/health</code> . <code>false</code> = legacy mode (migration only; warning at startup).
BRAINBOX_ADMIN_TOKEN	long random value	Enables <code>/api/admin/*</code> (header <code>X-Admin-Token</code>). Unset = admin API returns 503. Use ≥ 32 characters.
JWT_SECRET_KEY	long random value	Signs staff logins. The placeholder or anything under 16 characters logs a loud warning: anyone who knows it can forge staff sessions. Changing it signs everyone out.
JWT_ALGORITHM, JWT_EXPIRATION_HOURS	<code>HS256</code> , <code>24</code>	Listed in <code>.env.example</code> but fixed in code (HS256; the 24 h applies to legacy <code>/api/auth</code> tokens only).
LEGACY_DOC_AUDIENCE	<code>public</code>	Audience assumed for content without a label (old documents, <code>/api/ingest</code> without audience). Set <code>internal</code> to hide it from the public.
STAFF DASHBOARD		
STAFF_JWT_HOURS	<code>12</code>	Staff login lifetime.
LOGIN_MAX_FAILURES, LOGIN_WINDOW_MINUTES	<code>10</code> , <code>15</code>	Failed logins per email + IP per window before 429 (in memory, per worker process).

Variable	Example / default	Purpose
DASHBOARD_URL	<code>https://dashboard.example.com</code>	Base of invite / reset / alert links (no trailing slash). Unset: the request's Origin is used; alert emails then carry no link.
GAP_DISTANCE_THRESHOLD	<code>0.55</code>	Default gap threshold (each company can override it).
EMAIL_MAX_PER_HOUR	<code>20</code>	Default alert-email budget per company per hour.
EMAIL (SMTP). LEAVE SMTP_HOST EMPTY TO DISABLE ALL EMAIL		
SMTP_HOST, SMTP_PORT	<code>smtp.example.com</code> , <code>587</code>	Port 0/empty = 587 (starttls), 465 (ssl) or 25 (none).
SMTP_USER, SMTP_PASSWORD	<code>–</code>	Login, if the server needs one.
SMTP_FROM	<code>Brainbox <no-reply@example.com></code>	Sender. Email is "configured" when SMTP_HOST and SMTP_FROM (or SMTP_USER) are set.
SMTP_TLS, SMTP_TIMEOUT	<code>starttls</code> , <code>15</code>	<code>starttls ssl none</code> ; seconds.

Generate random values with `python -c "import secrets; print(secrets.token_urlsafe(48))"`.

Command line (on the server)

Run inside the brainBox folder (with Docker: `docker exec -i brainbox-api python -m app.cli ...`). The CLI talks to the database directly and needs no token.

```
# API keys
python -m app.cli keys create --tenant acme --type publishable --name "Website widget"
python -m app.cli keys create --tenant acme --type secret --name "Odoo server" --expires 2027-12-31
# import a key a live site already uses (reads it from stdin)
python -m app.cli keys create --tenant acme --type publishable --name "Legacy site" --key-stdin
python -m app.cli keys list [--tenant acme]
python -m app.cli keys revoke 42

# Staff users
python -m app.cli staff create --tenant acme --email owner@acme.com --role owner [--name "Ada"]
python -m app.cli staff create ... --password          # prompt for a password instead of an invite
python -m app.cli staff list [--tenant acme]
python -m app.cli staff set-password owner@acme.com    # also completes a pending invite
python -m app.cli staff deactivate owner@acme.com [--force]
```

`keys list` never prints raw keys; `keys create` prints the raw key once. `--password-stdin` reads a password from stdin; `deactivate --force` works even on the last owner. `staff create` without a password prints an invite link valid 7 days.

Deployment order (turning keys on safely)

- 1 Deploy with `REQUIRE_API_KEY=false`.** The new tables and columns are created on startup; existing clients keep working.
Set `JWT_SECRET_KEY`, `BRAINBOX_ADMIN_TOKEN`, `DASHBOARD_URL` and `SMTP_*` now too.
- 2 Register a key for every existing client.** Websites that already ship a key: import that exact key as publishable (`--key-stdin`). Servers (Odoo, backends, admin tools): create a secret key and put it in that server's settings, never in front-end code.
Check with `python -m app.cli keys list`.

- 3 **Decide** `LEGACY_DOC_AUDIENCE` and relabel sensitive sources (Training page or `PATCH /api/train/sources/{id}`).
- 4 **Create each company's first owner** (`staff create` or `POST /api/admin/staff`) so they can invite their team.
- 5 **Switch to** `REQUIRE_API_KEY=true` and restart. Watch the logs for 401/403; switching back to `false` rolls back instantly.

OPERATIONAL NOTES

- Training runs inside the API process: a restart marks running jobs as failed ("Interrupted by a server restart"). Re-upload or re-sync them.
- Login rate limiting is per worker process; with several workers, put a rate limit in your reverse proxy too.
- Reverse proxies in front of brainBox must allow requests of at least 200 s for `/api/chat` and uploads of 25 MB for `/api/train/file`.

14 Glossary

Term	Meaning
Admin token	One server-wide password (<code>BRAINBOX_ADMIN_TOKEN</code>) for managing keys of all companies and creating first owners.
Audience	Label on trained content saying who may see it: public, customer, vendor, internal or admin.
Cache	Brainbox remembers each answer for one hour per company, role and question, and forgets them all when training changes.
Chunk	A short passage (about 1,000 characters, or one support ticket) cut from your content. The unit Brainbox searches.
Cosine distance	How different two fingerprints are: 0 = same meaning, higher = less related. Used to find the closest chunks and to flag weak matches.
Data path	Where the list of records sits in an API response, written with dots: <code>data.tickets</code> .
Embedding	The "meaning fingerprint": 768 numbers describing what a chunk or question is about.
Gap (knowledge gap)	A question Brainbox couldn't answer well, collected in the inbox for staff to answer.
Ingest	The low-level way to push raw content (logs, code). Training sources are the friendlier way.
JWT	A signed login token. Staff logins are JWTs valid 12 hours.
LLM / AI model	The language model (Ollama, optionally OpenAI) that writes the answer from the chunks it is given.
Mapping	Telling Brainbox which field of a ticket is the id, title, issue, answer or conversation.
Pagination	How an API splits long lists into pages (page numbers or a "next" cursor).
Proxy	Your own server endpoint that forwards chat requests to brainBox with the secret key and the real user, so browsers never hold a key.
Publishable key	<code>pk_live_...</code> : may be public; chat only; users are treated as public.
Retrieval	Finding the closest chunks for a question (here: the top 5 the role may see).
Role	Who is asking (admin, internal, customer/portal, vendor, public); decides which audiences are searched. Not the same as a staff role.
Secret key	<code>sk_live_...</code> : full access for one company; servers only.
Session	One conversation (its questions and answers), optionally owned by a <code>user_id</code> .
Staff role	Dashboard permission level: owner, admin, trainer or viewer.
Sync	Fetching an API source again to pick up new and changed tickets.
Task	A background job (training or ingest) with an id you can poll.
Tenant	One client company's sealed space: its knowledge, conversations, staff, keys and settings.
Training source	One thing you trained on (a file, a text, an API, a gap answer), with a status and an audience.

WHERE TO GO NEXT

Owners and staff: sign in to the dashboard, upload your top FAQ document, and check the Knowledge gaps inbox after a week. Developers: start with recipe 6 (chat) or recipe 3 (support tickets), and use `GET /api/health` with your key as the connection test.